VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Deep Learning and Its Applications (CO3133)

Semester 261 – Class CC01 – Group G-02

Assignment 1

From Linear Models to Modern Sequence Models

A Comparative Study for Image Classification on Fashion-MNIST

Lecturer: Lê Thành Sách

Students: Võ Thị Như Quỳnh – 2353041
Mai Trung Kiên – 2352641
Hứa Tuệ Minh – 2450018

HO CHI MINH CITY, OCTOBER 2026

TABLE OF CONTENTS

1 Problem and data description **2**
1.1 Problem statement . . . 2
1.2 Dataset source and scope . . . 2
1.3 Read the dataset . . . 2
1.4 Define the train and validation subsets . . . 2
1.5 Image Visualization and Pixel Representation . . . 3
1.6 Class Distribution and Balance . . . 4
1.7 Representative Training Images . . . 4
1.8 Pixel Intensity and Spatial Distribution . . . 6
1.9 Brightness, Contrast and Foreground Coverage . . . 7
1.10 Average Appearance of Each Class . . . 10
1.11 PCA: How Much Variation Fits in Fewer Dimensions? . . . 12
1.12 Two-Dimensional Visualization: PCA vs. t-SNE . . . 13
1.13 Inspect Preprocessing . . . 13
1.14 EDA conclusions . . . 14

2 Methodology **15**
2.1 Pipeline . . . 15
2.2 Data and inputs . . . 15
2.3 Model Definitions . . . 16
2.4 Training and selection . . . 19
2.5 Loss, metrics and timing . . . 19
2.6 Experiment design . . . 20
2.7 Training and validation loop . . . 20
2.8 Reproduction and Recorded environment . . . 21

3 Implementation results **22**
3.1 Validation comparison . . . 22
3.2 Held-out test results . . . 23
3.3 Training-seed variability . . . 23
3.4 Learning curves . . . 24
3.5 Test confusion matrices . . . 26
3.6 Inspect predictions . . . 29
3.7 Error analysis . . . 30
3.8 Ablation: matched parameter budget . . . 32
3.9 Results discussion . . . 33
3.10 Post-results analysis . . . 33
3.11 Limitations and conclusion . . . 41

4 AI Usage Disclosure **42**

References **43**

1 Problem and data description

1.1 Problem statement

Predict one of ten clothing classes from a 28×28 grayscale image for automatic product categorization. Similar upper-body silhouettes and limited detail make some classes difficult to separate.

Given $x \in \mathbb{R}^{1\times28\times28}$, a model f_θ outputs ten class scores and predicts $\hat{y} = \arg\max_k f_\theta(x)_k$; each image is one prediction unit. We compare five architecture families (a linear classifier, an MLP, a CNN, an LSTM/GRU and a Transformer) under one shared data split and training protocol, measuring accuracy and macro-F1 against parameter count and runtime cost.

1.2 Dataset source and scope

We use the 2017 Fashion-MNIST release by Zalando Research, described by Xiao [1], Rasul and Vollgraf and distributed under the MIT license. It is our sole dataset; each product image has one supplied category label.

Fashion-MNIST contains 60,000 official training images and 10,000 test images, stored as IDX arrays with pixel values from 0 to 255. The classes are T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag and Ankle boot.

1.3 Read the dataset

The official Fashion-MNIST data are stored in IDX format, with 16-byte image headers and 8-byte label headers.

(a) Dataset arrays

	Train	Test
Images	60,000	10,000
Image size	28×28	28×28
Labels	60,000	10,000
Dtype	uint8	uint8
Range	0–255	0–255

(b) IDX files

File	MiB
Train images	44.861
Train labels	0.057
Test images	7.477
Test labels	0.010
Total	**52.404**

Table 1: Fashion-MNIST dataset structure and storage summary.

The dataset contains 60,000 training and 10,000 test images, each of size 28×28 with uint8 pixel values in $[0, 255]$. Image and label counts match, and no explicit channel dimension is present. The four uncompressed IDX files occupy 52.404 MiB, including their headers; the downloaded `.gz` files are compressed copies.

1.4 Define the train and validation subsets

We partition the original 60,000 training images into training (90%) and validation (10%) subsets using stratified sampling (`test_size=0.10`, `random_state=42`). All detailed EDA, fitted transformations, and visualization samples use only the training subset. Validation is used for model selection, the official test set remains separate for final evaluation.

	Train	Validation	Official test
Per class	5,400	600	1,000
Total	**54,000**	**6,000**	**10,000**

Classes (10): T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, Ankle boot.

Table 2: Stratified dataset split across 10 balanced classes.

1.5 Image Visualization and Pixel Representation

To examine the visual characteristics and numerical representation of Fashion-MNIST, we inspect an actual image from the training subset at its original resolution of 28×28 pixels.

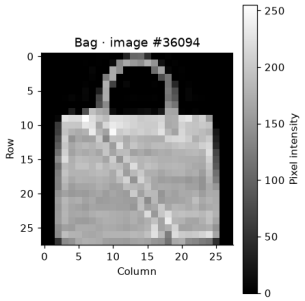


Figure 1: A training example displayed at its native 28×28 resolution, with its original dataset index and class label.

Each image is represented as a single-channel grayscale matrix, where pixel intensities range from 0 (black) to 255 (white). Darker pixels primarily represent the background, while brighter pixels describe the clothing item's shape and appearance. Due to the limited spatial resolution, fine textures and small visual details are difficult to distinguish. Moreover, the assigned label specifies a general clothing category rather than an individual product.

To further investigate the underlying numerical structure, we examine an 8×8 pixel patch and visualize the flattened representation of the same image.

Rows 10–17, columns 10–17

234	228	219	212	205	203	203	217
249	202	210	202	208	201	206	226
193	190	171	187	179	176	174	174
157	221	175	197	185	179	187	188
165	187	188	162	170	160	169	174
205	149	202	188	181	185	181	176
193	188	166	220	181	193	188	179
170	202	158	185	201	175	175	166

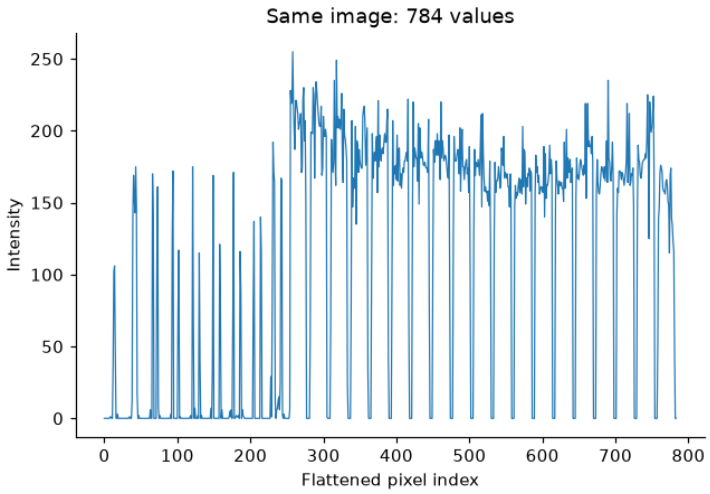


Figure 2: Pixel intensities in an 8×8 image patch and the corresponding image representation flattened into a 784-dimensional vector.

The selected patch contains intermediate intensity values, confirming that the image is not a binary silhouette but a grayscale representation with multiple intensity levels. For conventional machine learning models, the original two-dimensional image can be reshaped into a one-dimensional feature vector:

$$X \in \{0, \ldots, 255\}^{28 \times 28} \quad \longrightarrow \quad \mathbf{x} \in \{0, \ldots, 255\}^{784}.$$

This flattening operation preserves all 784 pixel values in row-major order without resizing or information loss. However, the resulting vector no longer explicitly represents the two-dimensional spatial relationships between neighboring pixels. Such spatial structure can instead be exploited directly by image-oriented architectures, such as convolutional neural networks (CNNs).

1.6 Class Distribution and Balance

We examine class frequencies in the original Fashion-MNIST training and test partitions to identify potential class imbalance. Both partitions contain the same ten clothing categories, with equal representation across classes. Official test labels are summarized descriptively only; test images are excluded from preprocessing fitting, model training, and model selection.

Class	Original train	Official test	Train share (%)	Test share (%)
T-shirt/top	6000	1000	10.0	10.0
Trouser	6000	1000	10.0	10.0
Pullover	6000	1000	10.0	10.0
Dress	6000	1000	10.0	10.0
Coat	6000	1000	10.0	10.0
Sandal	6000	1000	10.0	10.0
Shirt	6000	1000	10.0	10.0
Sneaker	6000	1000	10.0	10.0
Bag	6000	1000	10.0	10.0
Ankle boot	6000	1000	10.0	10.0

Table 3: Class distribution in the original Fashion-MNIST partitions.

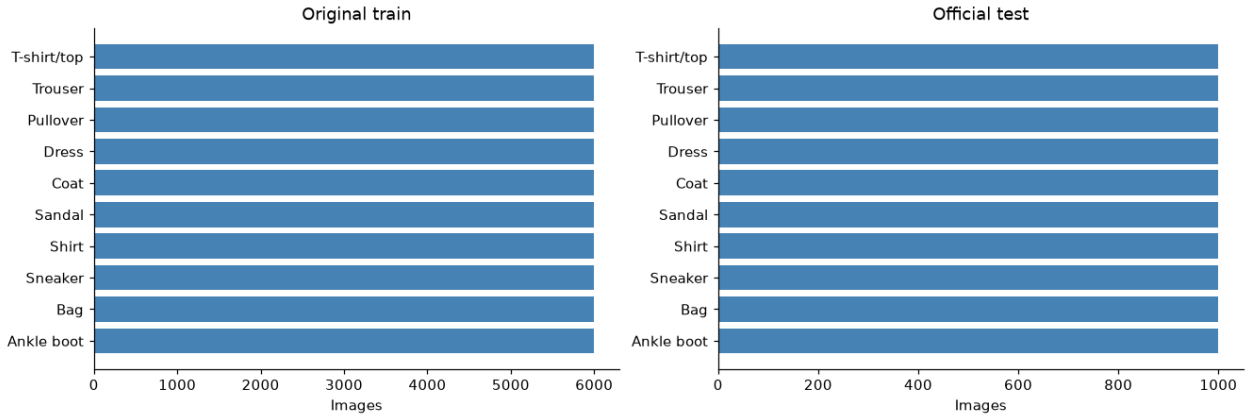


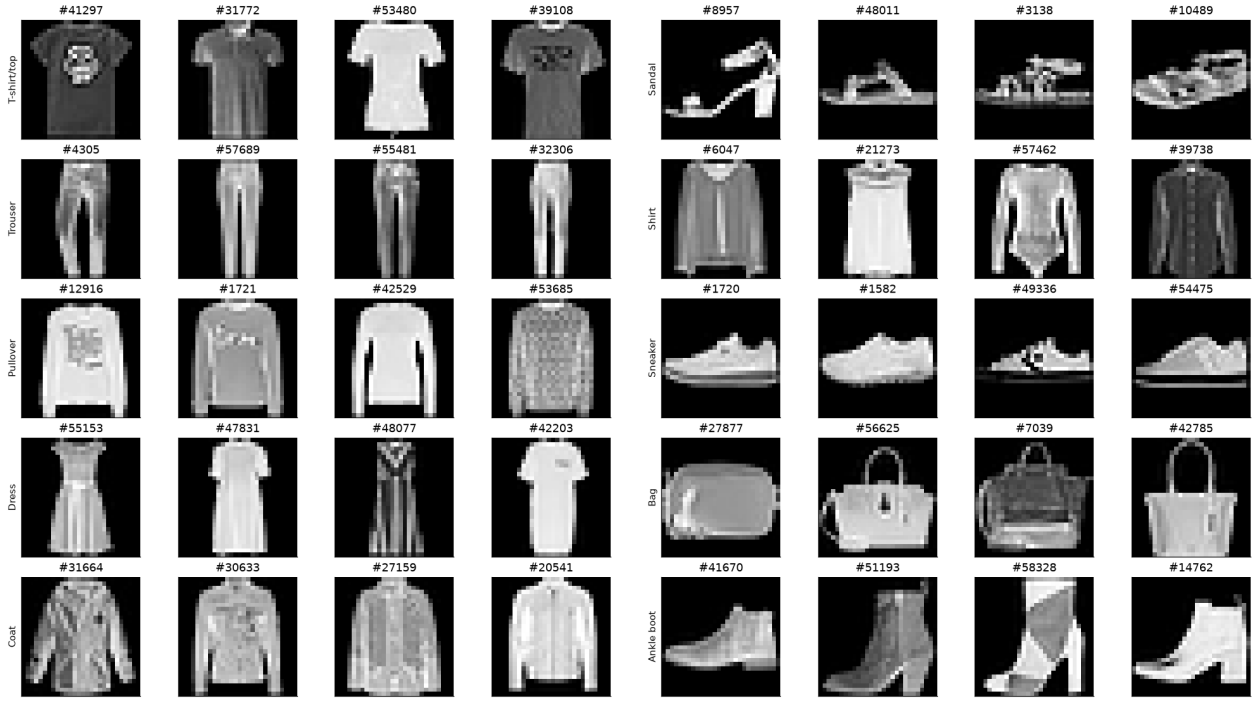
Figure 3: Class-frequency comparison between the original training and official test sets.

Each class accounts for exactly 10% of its respective partition, yielding a maximum-to-minimum class-frequency ratio of 1:1. The uniform bar heights within each panel confirm the absence of class-frequency imbalance. Because the panels use separate axes, comparisons should focus on relative frequencies rather than absolute bar lengths across panels.

1.7 Representative Training Images

To qualitatively examine the visual diversity of Fashion-MNIST, we select four examples from each of the ten classes using a fixed random seed. This produces 40 reproducible training samples, allowing us to inspect both within-class variation and visual similarities between different categories.

Figures below present the selected examples, grouped into five classes per figure.



(a) T-shirt/top, Trouser, Pullover, Dress, Coat

(b) Sandal, Shirt, Sneaker, Bag, Ankle boot

Figure 4: Representative Fashion-MNIST training images, showing four seeded examples per class across ten clothing categories (40 images in total).

Visual inspection reveals two important characteristics: *intra-class variation*, where images within the same category differ in appearance, and *inter-class similarity*, where different categories share visual features. Table 4 summarizes the main qualitative observations and their potential implications for classification.

Class / Group	**Visual Observation**	**ML Implication**
Trouser	Distinctive two-leg silhouette.	Potentially recognizable shape cues.
Dress	Differences in length and overall outline.	Within-class shape variation.
Pullover / Coat	Similar torso-and-sleeves structure.	Potential inter-class confusion.
T-shirt/top / Shirt	Overlapping upper-body shapes and details.	Requires more discriminative features.
Sandal / Sneaker / Ankle boot	Differences in foot and ankle coverage.	Footwear structure provides useful cues.
Bag	Variation in body shape and handle visibility.	Requires robustness to appearance variation.

Table 4: Qualitative visual observations and their potential implications for classification.

Overall, the selected examples illustrate substantial appearance variation within classes and shared visual structures across some categories. In particular, upper-body clothing classes may be difficult to distinguish using coarse shape information alone, whereas trousers exhibit a more distinctive silhouette.

These observations suggest that successful classification requires features that capture both general shape and finer visual details. However, four examples per class provide only a qualitative overview and cannot establish the frequency of particular appearances or actual misclassification rates. Such hypotheses must be examined quantitatively through class-wise evaluation and confusion-matrix analysis.

1.8 Pixel Intensity and Spatial Distribution

We analyze pixel intensities across 54,000 training images to examine their distribution and spatial variation.

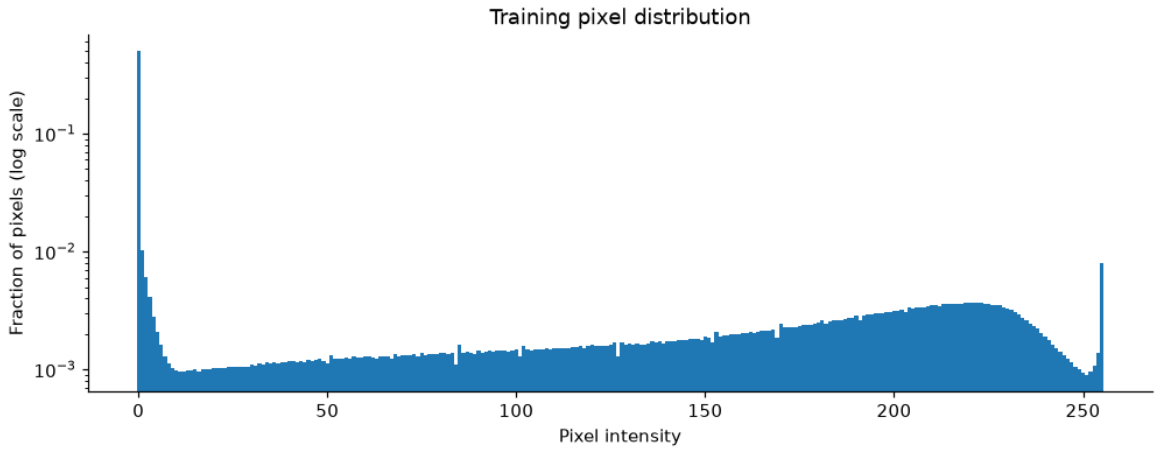


Figure 5: Training pixel-intensity distribution (0–255) on a logarithmic vertical scale.

The histogram shows a dominant peak at zero, reflecting dark background regions. The logarithmic scale reveals less frequent intensities, confirming that foreground pixels contain multiple grayscale levels.

Statistic	**Value**
Zero-valued pixels	50.2143%
Completely black images	0

Table 5: Training pixel-intensity statistics.

Although many pixels represent the background, no training image is entirely black. To investigate spatial patterns, we compute per-pixel mean and variance maps using intensities normalized to $[0, 1]$.

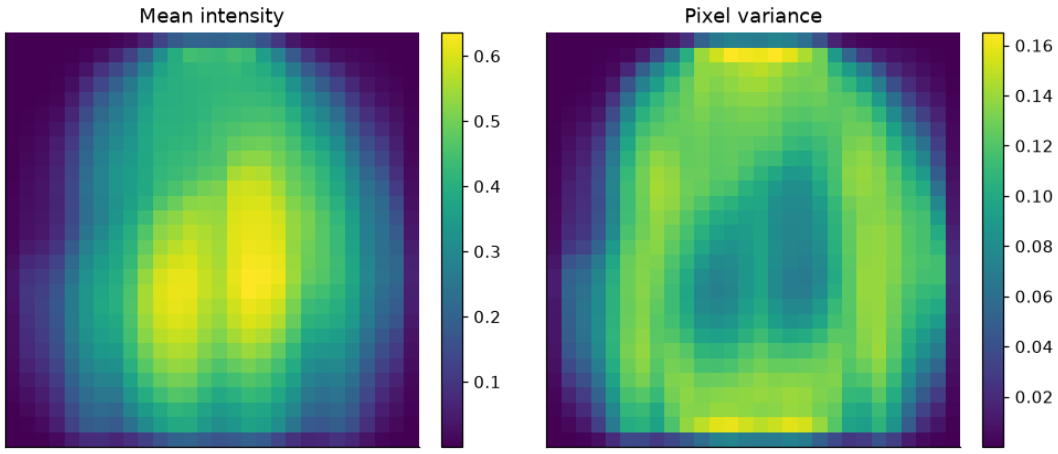


Figure 6: Per-pixel mean and variance maps across the training subset using normalized intensities.

The mean map indicates typical clothing locations, while variance highlights regions with greater intensity changes. Background corners remain relatively stable. However, low variance does not necessarily imply low discriminative value, so these observations alone do not justify feature removal.

1.9 Brightness, Contrast and Foreground Coverage

We characterize each of the 54,000 training images using three global features: brightness (mean intensity), contrast (intensity standard deviation), and foreground coverage (fraction of pixels exceeding a fixed threshold).

For an image with $P = 28 \times 28 = 784$ pixels, let I_p be the raw intensity and $x_p = I_p/255$ its normalized value. Brightness and contrast are defined as

$$B = \frac{1}{P}\sum_{p=1}^{P} x_p,$$

$$C = \sqrt{\frac{1}{P}\sum_{p=1}^{P}(x_p - B)^2}.$$

Foreground coverage at raw threshold $\tau = 26$ is

$$F_\tau = \frac{1}{P}\sum_{p=1}^{P} \mathbf{1}\{I_p > \tau\}.$$

Statistic	Brightness	Contrast	Coverage
Mean	0.286	0.320	0.451
Std. dev.	0.126	0.078	0.149
Minimum	0.019	0.070	0.038
5th percentile	0.101	0.178	0.214
Median	0.272	0.332	0.458
95th percentile	0.503	0.428	0.673
Maximum	0.752	0.475	0.909

Table 6: Distributions of image-level appearance features over 54,000 training images; standard deviations are computed across images.

These features describe global appearance, not image correctness. Brightness depends on both intensity and occupied area, while coverage depends on the threshold. Extreme values require visual inspection before cleaning.

1.9.1 Compare Appearance by Class

We compare feature distributions across the ten classes using boxplots. Outlier markers are hidden for readability, but all observations remain included in the calculations.

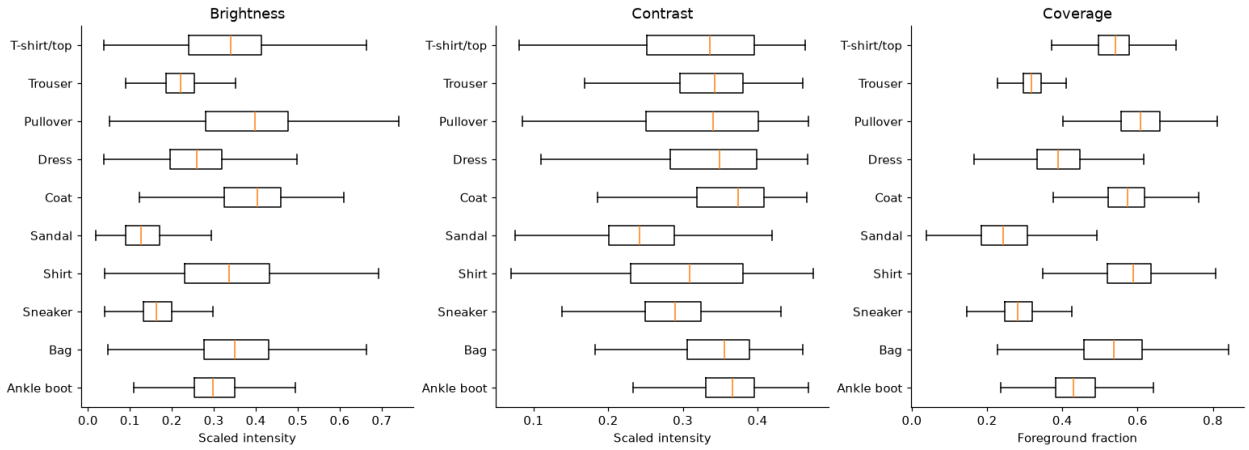


Figure 7: Class-wise distributions of brightness, contrast, and foreground coverage.

The distributions overlap, so no single feature clearly separates all classes. Sandals generally have lower coverage than fuller garments, while brightness differences may reflect both shape and background.

1.9.2 Inspect the Extreme Examples

For each feature, we inspect the two lowest and two highest training examples to identify unusual appearances.

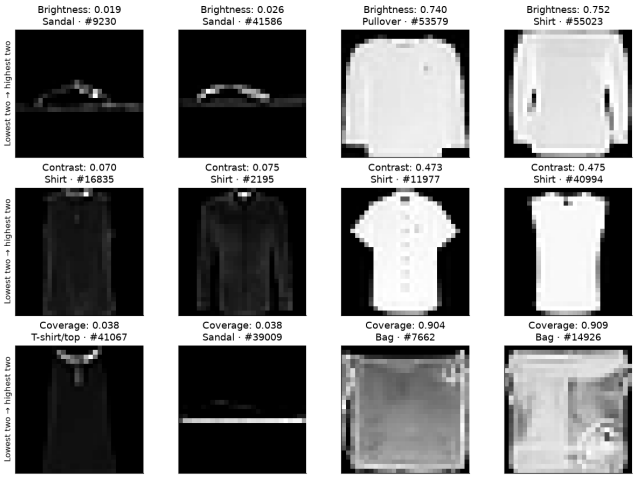


Figure 8: Two lowest and two highest examples for each appearance statistic.

The darkest examples are sandals, the lowest-contrast examples are shirts, and the highest-coverage examples are bags. Sandal #39009 contains an unusual bright region below the footwear. These selected extremes warrant inspection but are neither representative samples nor confirmed labeling errors.

1.9.3 What Does the Foreground Threshold Select?

A binary threshold mask identifies pixels whose raw intensity exceeds $\tau = 26$:

$$M_\tau(I_p) = \begin{cases} 1, & I_p > \tau, \\ 0, & I_p \le \tau. \end{cases} \qquad \tau = 26.$$

The foreground coverage is the proportion of pixels selected by this mask:

$$F_\tau = \frac{1}{784} \sum_{p=1}^{784} M_\tau(I_p).$$

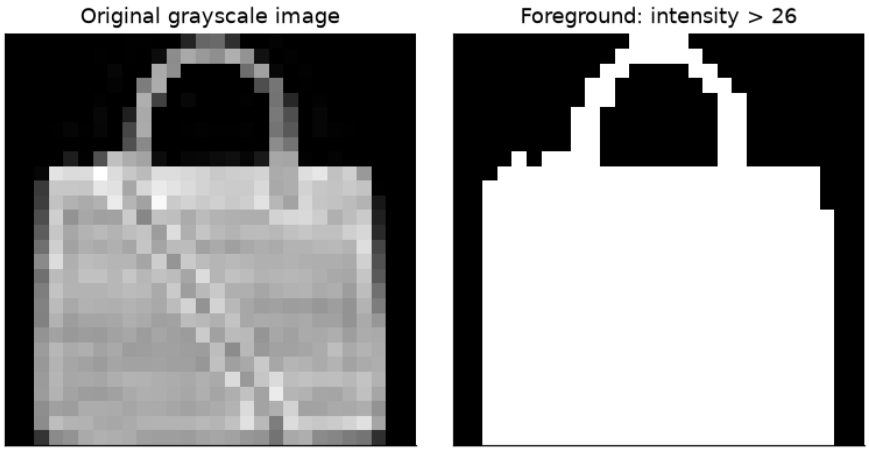


Figure 9: Original grayscale image and its threshold-based foreground mask ($\tau = 26$).

Thresholding separates occupied and background pixels but may exclude dark garment regions. Thus, coverage is an approximate foreground measure, not precise segmentation. Original grayscale images remain unchanged.

1.9.4 Sensitivity to the Foreground Threshold

To assess threshold sensitivity, we compare class-wise median coverage at raw thresholds $\tau \in \{16, 26, 36\}$.
For any fixed image, coverage is non-increasing as the threshold increases:

$$F_{16} \geq F_{26} \geq F_{36}.$$

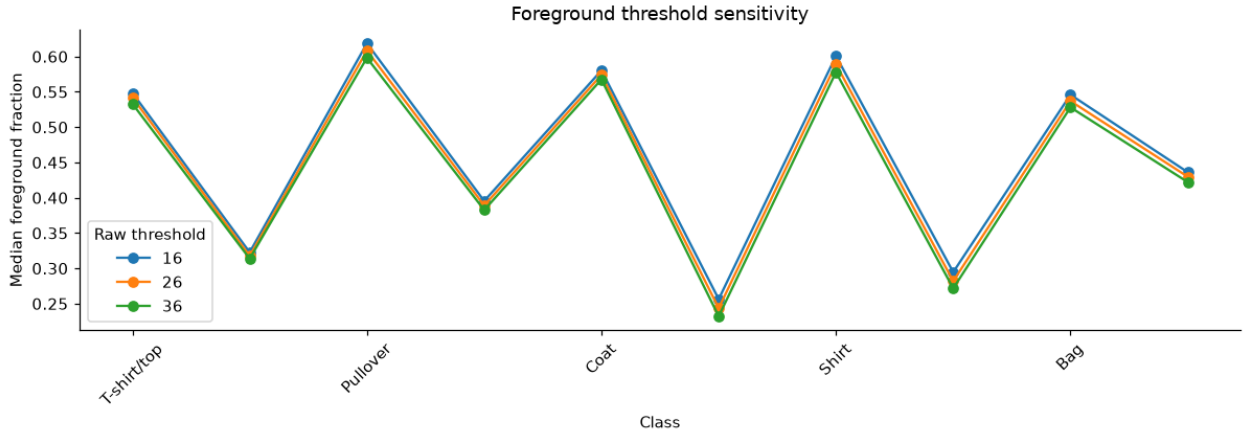


Figure 10: Sensitivity of class-wise median foreground coverage to thresholds 16, 26, and 36.

Higher thresholds reduce or preserve measured coverage. Changes in the spacing or ordering of class curves indicate sensitivity to the threshold choice. All thresholds are applied to the same unmodified images, not alternative cleaned datasets.

1.10 Average Appearance of Each Class

To identify common visual patterns, we compute the mean image of each class using normalized training images. For class c with N_c images, the pixel-wise mean is

$$\mu_c(u, v) = \frac{1}{N_c} \sum_{i:y_i=c} X_i(u, v), \qquad X_i(u, v) \in [0, 1].$$

Here, (u, v) denotes a pixel coordinate and N_c is the number of training images in class c.

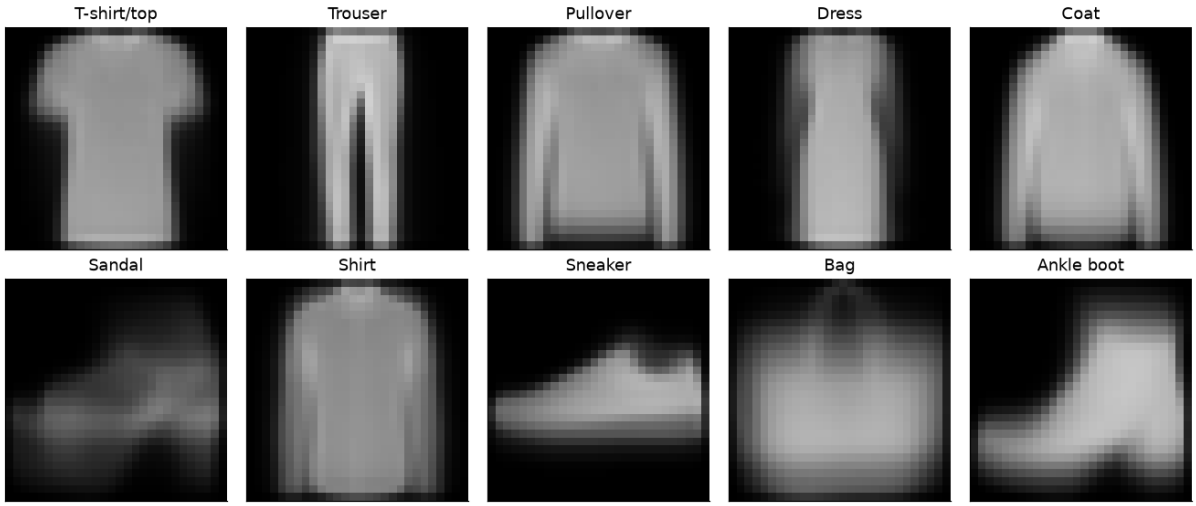


Figure 11: Mean training image for each of the ten classes, displayed on a common grayscale scale.

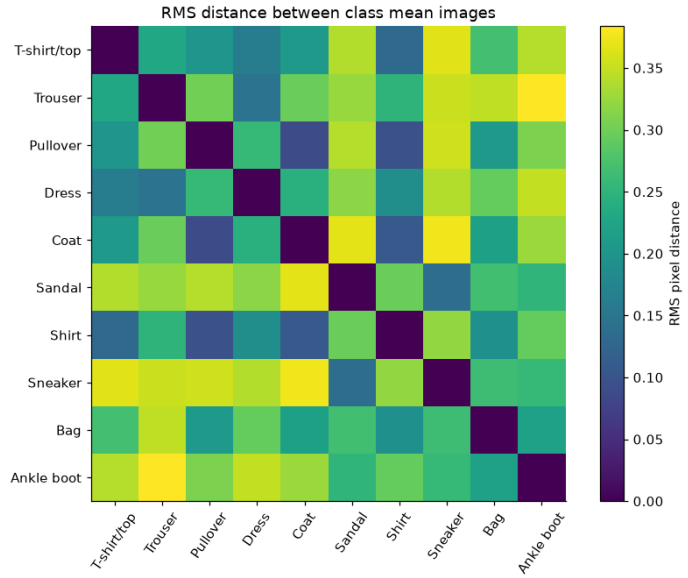
Averaging preserves common silhouettes while smoothing individual details. Trousers and footwear retain distinctive shapes, whereas several upper-body classes share similar outlines. These mean images are statistical summaries, not actual clothing examples.

1.10.1 Similarity Between Class Means

We quantify the similarity between class means using root-mean-square (RMS) distance on normalized intensities:

$$D_{\text{RMS}}(a, b) = \sqrt{\frac{1}{784} \sum_{u=1}^{28} \sum_{v=1}^{28} [\mu_a(u, v) - \mu_b(u, v)]^2}.$$

Smaller distances indicate more similar average pixel patterns.



(a) Pairwise RMS distance matrix

Class pair	RMS
Pullover – Coat	0.088
Pullover – Shirt	0.096
Coat – Shirt	0.107
T-shirt/top – Shirt	0.132
Sandal – Sneaker	0.137

(b) Five closest class-mean pairs

Figure 12: Similarity between Fashion-MNIST class mean images: (a) pairwise RMS distance matrix and (b) the five smallest distances between distinct classes.

Pullover and Coat have the closest mean images ($D_{\text{RMS}} = 0.088$), consistent with their shared torso-and-sleeve structure. However, mean-image similarity does not directly measure classification difficulty or prediction errors.

1.10.2 Visual Inspection of the Closest Pair

To examine the closest pair beyond their averages, we revisit the previously sampled Pullover and Coat examples.

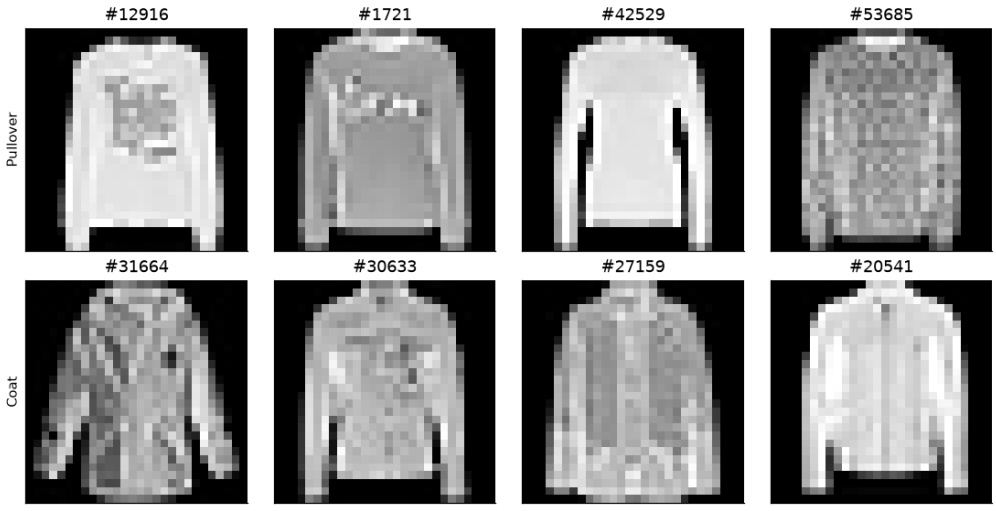


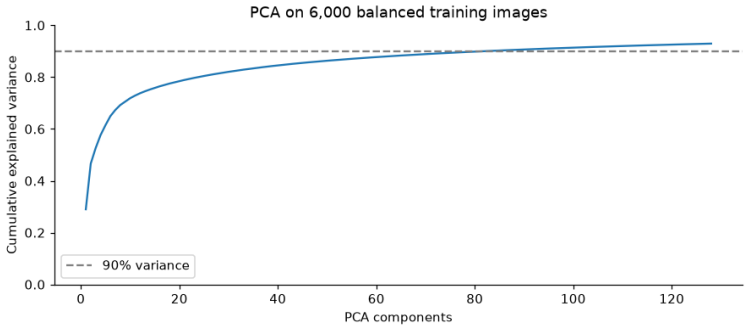
Figure 13: Individual Pullover and Coat examples, the two classes with the closest mean images.

The examples reveal both shared silhouettes and details obscured by averaging, including differences in necklines, openings, garment length, and texture. Such distinctions may be subtle at 28×28 resolution. Therefore, similarity between class means suggests potential visual ambiguity but does not establish actual model confusion.

1.11 PCA: How Much Variation Fits in Fewer Dimensions?

We fit Principal Component Analysis (PCA) on a balanced sample of 6,000 training images. Class labels are used only for sampling; PCA itself is unsupervised and identifies directions of maximum pixel variance. For eigenvalues $\lambda_1 \geq \cdots \geq \lambda_{784}$ of the sample covariance matrix, the cumulative explained variance of the first k components is

$$\mathrm{CEV}(k) = \frac{\sum_{j=1}^{k} \lambda_j}{\sum_{j=1}^{784} \lambda_j} \times 100\%.$$



(a) Cumulative explained variance curve

PCs	Variance (%)
2	46.62
8	69.24
16	76.45
32	82.61
64	88.22
128	92.94

(b) Selected PCA component counts

Figure 14: PCA explained variance on 6,000 training images: (a) cumulative variance curve and (b) variance retained at selected dimensions.

The first two components retain only 46.62% of total variance, while 128 retain 92.94%. The curve gradually flattens, indicating diminishing variance contributions from later components. Since only 128 components were computed, higher variance targets cannot be resolved from this run. Retained variance alone does not establish classification performance. To visualize information loss, we reconstruct sampled images using different numbers of principal components. For a normalized image vector $\mathbf{x}$, the rank-k reconstruction is

$$\hat{\mathbf{x}}_k = \boldsymbol{\mu} + V_k V_k^\top (\mathbf{x} - \boldsymbol{\mu}),$$

where $\boldsymbol{\mu}$ is the training-sample mean and V_k contains the first k principal directions. Only displayed intensities are clipped to $[0, 1]$.

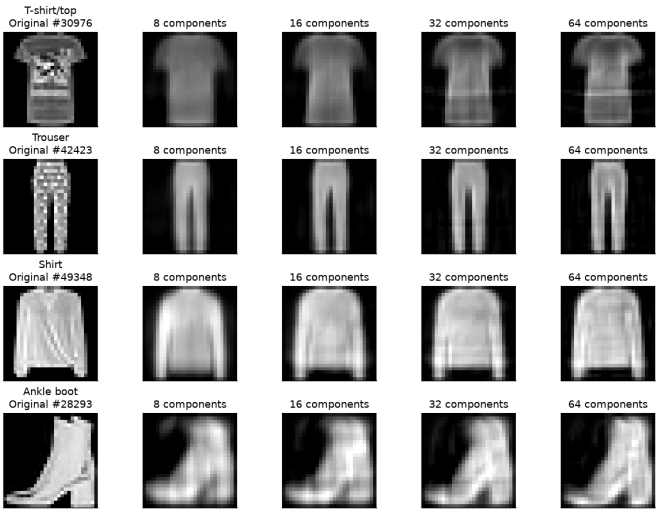
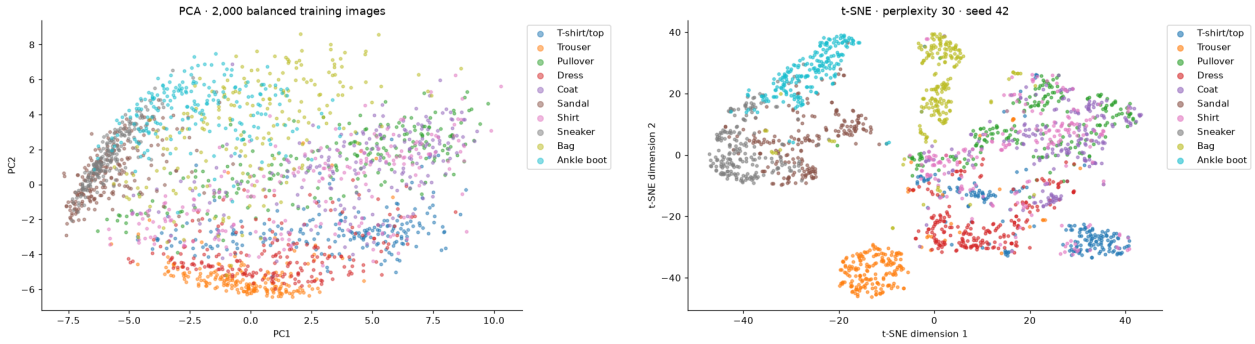


Figure 15: PCA reconstructions at four ranks for images from the PCA fitting sample.

Broad silhouettes remain recognizable with eight components, while higher ranks recover sharper structural details. However, the patterned trousers and printed T-shirt still lose fine details at 64 components. These reconstructions illustrate information retention within the fitting sample, not generalization to unseen images.

1.12 Two-Dimensional Visualization: PCA vs. t-SNE

We select 200 images per class (2,000 total) from the balanced PCA sample to compare two-dimensional projections. PCA uses the first two principal-component scores, whereas t-SNE embeds the same images using their first 50 PCA scores. Both methods are used solely for exploratory visualization, not classifier training.



(a) PCA: first two components

(b) t-SNE: first 50 PCA scores

Figure 16: Two-dimensional projections of the same 2,000 training images (200 per class): (a) PCA and (b) t-SNE applied to 50-dimensional PCA representations.

Aspect	PCA	t-SNE
Method	Linear projection	Nonlinear embedding
Input	Original pixel features	First 50 PCA scores
Objective	Maximize retained variance	Preserve local neighborhood similarities
Observed pattern	Considerable class overlap	Local clusters; some class mixing
Interpretation	Global variance directions	Local neighborhood structure
Limitation	2D projection may hide class separation	Global distances and cluster sizes are unreliable

Table 7: Comparison of PCA and t-SNE for exploratory visualization of the same training sample.

PCA shows substantial overlap between clothing categories, as its leading directions capture overall pixel variance rather than class separation. In the t-SNE view, trousers form a compact neighborhood, while Pullover, Coat, and Shirt remain mixed. Footwear forms another region, with some overlap between Sandal and Sneaker. These patterns suggest visually similar categories worth further investigation. However, neither projection measures classification accuracy: PCA overlap does not imply inseparability in the original feature space, and t-SNE distances between distant clusters are not reliable global similarity measures. Both visualizations depend on the selected sample and projection settings.

1.13 Inspect Preprocessing

We compute normalization statistics using only the 54,000 training images to prevent data leakage. Pixel values are first scaled to $[0, 1]$, then standardized using the training-set mean and population standard deviation:

$$x_{\text{scaled}} = \frac{x_{\text{raw}}}{255}, \qquad x_{\text{std}} = \frac{x_{\text{scaled}} - \mu_{\text{train}}}{\sigma_{\text{train}}}$$

where $\mu_{\text{train}} = 0.286209$ and $\sigma_{\text{train}} = 0.353160$ are computed across all training pixels.

Representation	Mean	Std.	Min	Max
Raw	72.983297	90.055735	0.000000	255.000000
Scaled	0.286209	0.353160	0.000000	1.000000
Standardized	0.000000	1.000000	-0.810424	2.021156

Table 8: Pixel statistics before and after preprocessing (population standard deviation).

Standardization yields zero mean and unit variance over the entire training subset, not per image. Validation and test images reuse the same training-derived statistics.

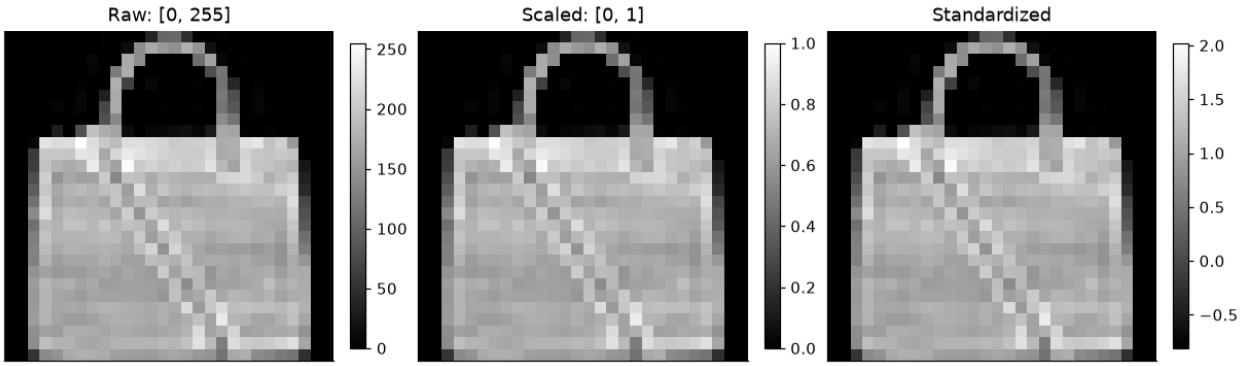


Figure 17: The same image under raw, scaled, and standardized representations, shown with corresponding display ranges.

All three representations preserve the image geometry and relative pixel contrast. Only numerical values change, as indicated by the color bars. Unlike scaling, standardization permits negative values and is not bounded by $[0, 1]$.

1.14 EDA conclusions

Evidence from the exploration	Implication
Ten equally frequent classes in the original sets and stratified subsets	Start from an unweighted baseline; inspect class-specific performance later.
Fixed 28×28 grayscale images	Keep native resolution; a flattened image contains 784 values.
Large background regions and varied foreground intensities	Scaling is useful; zero pixels alone are not a reason to reject an image.
Broad brightness and coverage distributions within classes	Inspect unusual examples visually before proposing cleaning rules.
Pullover and Coat have similar mean silhouettes	Treat related upper-body categories as a hypothesis for later error analysis.
Two-dimensional projections discard information and mix some labels	Use them for exploration, not as an estimate of classifier performance.
Normalization statistics are calculated from training pixels only	Reuse the same mean and standard deviation for later validation and test inputs.

Limits. The galleries are small samples. Pixel distances and threshold masks describe appearance, not product identity or labeling correctness. PCA and t-SNE are sample-based exploratory views. This exploration provides no classification accuracy or evidence about which trained model is best.

Preprocessing decision. Separate training code can reproduce the split using seed 42, `test_size=0.10` and `stratify=y`, and compute normalization on that training subset. The EDA supports keeping all images and all pixel coordinates for an initial baseline.

2 Methodology

2.1 Pipeline

All five models share one data pipeline, one training loop and one evaluation protocol; only the model block in Figure 18 changes between runs.

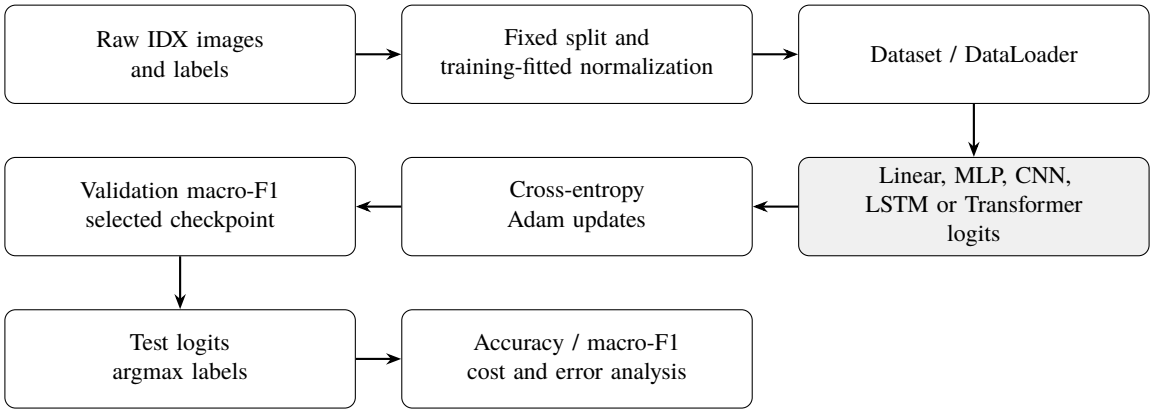


Figure 18: Shared training and evaluation pipeline. Only the shaded model block changes between runs.

We implemented IDX parsing, the `FashionMNISTDataset` class, all five model classes, the training and validation loop, checkpoint selection, evaluation and timing. PyTorch provides the layers (including `nn.LSTM` and `nn.TransformerEncoderLayer`), automatic differentiation, Adam and `DataLoader`; NumPy handles arrays; scikit-learn supplies `train_test_split` and the accuracy and F1 functions; Matplotlib produces the figures. No pretrained weights are used.

2.2 Data and inputs

The split always uses `random_state=42`, independently of the training seed, so every run sees the same 54,000 training, 6,000 validation and 10,000 test images. No images are removed or relabeled, and no augmentation is applied, except for the augmented extension models (Section 3.10.5).

Checkpoints store disjoint training/validation indices. Test data neither selects weights nor fits normalization. Product identifiers are unavailable, so image-index separation cannot establish product-level independence.

```
x = (pixel / 255 - training_mean) / training_std
training_mean = 0.286209    training_std = 0.353160

images: float32 [B, 1, 28, 28]
labels: int64   [B]          values 0 … 9
logits: float32 [B, 10]      output of every model

Linear, MLP:  flatten            -> [B, 784]
CNN:          image grid         -> [B, 1, 28, 28]
LSTM:         28 rows x 28 px    -> [B, 28, 28]
Transformer:  49 patches of 4x4  -> [B, 49, 16]
```

Training-only statistics normalize images once in the dataset constructor. Batch size is 128, with partial batches retained. Only training shuffles, using a seeded generator; loaders use zero workers. Inputs retain their native resolution and all pixel coordinates. `FashionMNISTDataset` implements `__len__` and `__getitem__` to supply normalized image-label pairs. An example full batch contains float32 images [128, 1, 28, 28] and int64 labels [128]; model logits have shape [128, 10]. Each model then reshapes the same normalized tensor into its own input representation, shown in the last four lines above.

2.3 Model Definitions

Let B denote batch size. All five models output ten raw logits; softmax is used only to inspect probabilities, not before cross-entropy loss. Table 9 compares their architectural assumptions and trade-offs.

Model	Inductive bias	Expected strength / limitation	Params
Linear	Linear pixel-to-class mapping; no spatial prior	Simple and efficient / no nonlinear interactions	7,850
MLP	Nonlinear fully connected features; no explicit locality	Flexible / spatial relations must be learned	203,530
CNN	Local filters, weight sharing, and pooling	Reusable spatial patterns / limited initial receptive field	105,866
LSTM	Sequential image rows with recurrent memory	Models row dependencies / compresses sequence into final state	24,714
Transformer	Patch tokens, learned positions, and global attention	Direct patch interactions / limited built-in spatial prior	19,530

Table 9: Inductive biases, architectural trade-offs, and parameter counts of the five implemented models.

2.3.1 Linear: A Weighted Pixel Baseline

Following the log-linear classification framework [2], each class receives a weighted sum of 784 pixel features plus a bias:

$$\mathbf{z} = W\mathbf{x} + \mathbf{b}.$$

Without hidden layers, the classifier cannot represent nonlinear pixel interactions.

```python
class LinearClassifier(nn.Module):
    # one score per class from the raw pixel vector
    def __init__(self):
        super().__init__()
        self.layers = nn.Sequential(
            nn.Flatten(), nn.Linear(784, 10)
        )

    def forward(self, images):
        return self.layers(images)
```

```
input  [B, 1, 28, 28]
  │
  │ Flatten        [B, 784]
  │ Linear 784→10  [B, 10]
  ▼
output [B, 10]

parameters = 784 × 10 + 10 = 7,850
```

2.3.2 MLP: Nonlinear Pixel Representations

The MLP learns hidden representations through backpropagation [3]. A 256-unit hidden layer uses the ReLU activation [8]:

$$\mathbf{h} = \mathrm{ReLU}(W_1\mathbf{x} + \mathbf{b}_1), \qquad \mathbf{z} = W_2\mathbf{h} + \mathbf{b}_2.$$

The nonlinear mapping supports complex decision boundaries, but flattened inputs lack an explicit spatial inductive bias.

```python
class MLP(nn.Module):
    # a nonlinear baseline without spatial assumptions
    def __init__(self):
        super().__init__()
        self.layers = nn.Sequential(
            nn.Flatten(),
            nn.Linear(784, 256),
            nn.ReLU(),
            nn.Linear(256, 10),
        )

    def forward(self, images):
        return self.layers(images)
```

```
input  [B, 1, 28, 28]
  │
  │ Flatten         [B, 784]
  │ Linear 784→256  [B, 256]
  │ ReLU            [B, 256]
  │ Linear 256→10   [B, 10]
  ▼
output [B, 10]

parameters = (784 × 256 + 256)
           + (256 × 10 + 10)
           = 203,530
```

2.3.3 CNN: Local Patterns with Shared Filters

CNNs exploit local receptive fields and shared filters to learn spatial features [4]. Our model uses two 3×3 convolution–ReLU– pooling blocks followed by a dense classifier. Padding preserves spatial dimensions during convolution, while 2×2 max pooling halves their size.

```python
class CNN(nn.Module):
    # shared filters preserve local image structure
    def __init__(self):
        super().__init__()
        self.layers = nn.Sequential(
            nn.Conv2d(1, 16, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(16, 32, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Flatten(),
            nn.Linear(32 * 7 * 7, 64),
            nn.ReLU(),
            nn.Linear(64, 10),
        )

    def forward(self, images):
        return self.layers(images)
```

```
input  [B, 1, 28, 28]
  |
  | Conv2d 1→16, 3×3   [B, 16, 28, 28]
  | ReLU               [B, 16, 28, 28]
  | MaxPool2d 2        [B, 16, 14, 14]
  | Conv2d 16→32, 3×3  [B, 32, 14, 14]
  | ReLU               [B, 32, 14, 14]
  | MaxPool2d 2        [B, 32, 7, 7]
  | Flatten            [B, 1568]
  | Linear 1568→64     [B, 64]
  | ReLU               [B, 64]
  | Linear 64→10       [B, 10]
  ▼
output [B, 10]

parameters = 105,866
```

2.3.4 LSTM: Reading the Image Row by Row

LSTM uses gated recurrent memory to model sequential dependencies [6]. We treat each image as 28 time steps, where each step is a 28-dimensional pixel row $\mathbf{r}_t$.

A single-layer, unidirectional LSTM with hidden size 64 updates hidden and cell states:

$$(\mathbf{h}_t, \mathbf{c}_t) = \text{LSTMCell}(\mathbf{r}_t, \mathbf{h}_{t-1}, \mathbf{c}_{t-1}).$$

The final hidden state is classified as

$$\mathbf{z} = W\mathbf{h}_{28} + \mathbf{b}.$$

Recurrent processing captures dependencies between rows, while horizontal structure is represented through features within each row.

```python
class LSTMClassifier(nn.Module):
    # image rows form a sequence from top to bottom
    def __init__(self):
        super().__init__()
        self.lstm = nn.LSTM(
            input_size=28,
            hidden_size=64,
            num_layers=1,
            batch_first=True,
        )
        self.classifier = nn.Linear(64, 10)

    def forward(self, images):
        rows = images.squeeze(1)
        _, (hidden, _) = self.lstm(rows)
        return self.classifier(hidden[-1])
```

```
input  [B, 1, 28, 28]
  |
  | squeeze(1)     [B, 28, 28]
  | LSTM 28→64     h: [1, B, 64]
  | hidden[-1]     [B, 64]
  | Linear 64→10   [B, 10]
  ▼
output [B, 10]

parameters = 24,714
```

2.3.5 Transformer: Attention Between Image Patches

Inspired by Transformer self-attention [7] and patch-based image representations [9], we divide each image into 49 non-overlapping 4×4 patches.

Each flattened patch $\mathbf{p}_j \in \mathbb{R}^{16}$ is projected to a 32-dimensional token with a learned positional embedding $\mathbf{q}_j$:

$$\mathbf{e}_j = W_p \mathbf{p}_j + \mathbf{b}_p + \mathbf{q}_j.$$

Two post-norm encoder layers use four attention heads, a feed-forward width of 64, ReLU activation, and no dropout. Scaled dot-product attention is defined as

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V, \qquad d_k = \frac{32}{4} = 8.$$

The 49 output tokens are mean-pooled and classified without a class token. Self-attention permits interactions between all patches, while spatial positions are supplied through learned embeddings.

```python
class TransformerClassifier(nn.Module):
    # attention combines patches with learned spatial positions
    def __init__(self):
        super().__init__()
        self.projection = nn.Linear(16, 32)
        self.positions = nn.Parameter(
            torch.empty(1, 49, 32)
        )
        nn.init.normal_(self.positions, std=0.02)
        layer = nn.TransformerEncoderLayer(
            d_model=32,
            nhead=4,
            dim_feedforward=64,
            dropout=0.0,
            batch_first=True,
        )
        self.encoder = nn.TransformerEncoder(
            layer, num_layers=2, enable_nested_tensor=False
        )
        self.classifier = nn.Linear(32, 10)

    def forward(self, images):
        patches = images.unfold(2, 4, 4).unfold(3, 4, 4)
        patches = patches.reshape(images.shape[0], 49, 16)
        tokens = self.projection(patches) + self.positions
        return self.classifier(
            self.encoder(tokens).mean(dim=1)
        )
```

```
input  [B, 1, 28, 28]
  |
  | unfold 4×4, twice  [B, 1, 7, 7, 4, 4]
  | reshape            [B, 49, 16]
  | Linear 16→32       [B, 49, 32]
  | + positions        [B, 49, 32]
  | encoder ×2         [B, 49, 32]
  | mean(dim=1)        [B, 32]
  | Linear 32→10       [B, 10]
  ▼
output [B, 10]

parameters = 19,530
```

The parameter total follows from the layer sizes:

```
parameters:
  Patch projection (16 -> 32):16 x 32 + 32 = 544
  Positional embeddings (49 patches, 32 dims):49 x 32 = 1,568
  Each Transformer encoder layer:
    Multi-head attention (Q, K, V, output): 4 x (32 x 32 + 32) = 4,224
    Feed-forward network (32 -> 64 -> 32): (32 x 64 + 64) + (64 x 32 + 32) = 4,192
    Two LayerNorms: 2 x (32 + 32) = 128
    Encoder total: 4,224 + 4,192 + 128 = 8,544
    Two encoder layers: 2 x 8,544 = 17,088
    Classifier (32 -> 10): 32 x 10 + 10 = 330
Total parameters: 544 + 1,568 + 17,088 + 330 = 19,530
```

2.4 Training and selection

All models use the settings below. They were fixed before training and shared across architectures; no hyperparameter search was performed on the validation or test data.

Setting	Value
Random seeds	42, 43, 44, 45, 46; each seed sets Python, NumPy and PyTorch and the training-shuffle generator. The data split stays fixed at `random_state=42`.
Optimizer	Adam, learning rate 0.001
Budget	16 epochs, batch size 128; every epoch runs (no early stopping)
Gradient clipping	Maximum total gradient norm 5.0
Selection	Highest validation macro-F1; earliest epoch wins an exact tie
Regularization	No augmentation, dropout, weight decay or learning-rate scheduler (the augmented extension models add augmentation, Section 3.10.5)
Execution	CPU, four PyTorch threads, float32 (no mixed precision), deterministic algorithms enabled

Each batch clears gradients, computes cross-entropy, backpropagates, clips gradients and updates with Adam. Non-finite losses or gradients raise errors. Validation follows each epoch in evaluation mode without gradients.

Reported metrics use selected weights in `best.pt`; `checkpoint.pt` stores epoch-16 weights. Test results do not select checkpoints.

History records predictions while weights change. Checkpoint training metrics use fixed selected weights, so they may differ from that epoch's history.

2.5 Loss, metrics and timing

For logits z and target y, cross-entropy is $\ell(z, y) = -z_y + \log \sum_{k=1}^{10} \exp(z_k)$. Batch loss is averaged; epoch loss weights batches by image count. Prediction uses $\arg\max_k z_k$ without post-processing.

- **Accuracy:** correct predictions divided by the number of images.
- **Macro-F1:** unweighted mean of the ten per-class scores, $F_1 = 2TP/(2TP + FP + FN)$; undefined scores are zero. It is the checkpoint-selection metric because it keeps weak classes visible.
- **Parameters:** number of trainable values in the model.
- **Training time:** the 16-epoch training and validation loop, including history writes and copies of best weights. It excludes dataset preparation, post-training fixed-weight metrics and final artifact generation.
- **Inference time:** the median of three complete batched passes after one warmup, divided by image count. It includes DataLoader iteration, the forward pass and argmax; it excludes metric calculation. It measures batched CPU throughput rather than single-request latency.
- **Aggregation:** each value is reported as mean $\pm$ sample standard deviation over the five seeds.

2.6 Experiment design

Under one shared protocol, how does the way an image is represented, and the inductive bias that representation allows, affect accuracy, macro-F1 and cost? Two experiments address this question (Table 10). Section 3.10 reports the post-results analysis: robustness and interpretability reuse the saved checkpoints; GRU, the representation ablation and shift + noise augmentation are new models trained with the same protocol.

	Experiment 1: main comparison	**Experiment 2: matched parameter budget (ablation)**
Hypothesis	Nonlinear features improve on the linear baseline; models that keep spatial arrangement (CNN, LSTM rows, Transformer patches with positions) use their parameters more efficiently than flattened-input models.	If an advantage comes from inductive bias rather than size, it remains when all models have about 25k parameters.
Changed factor	Architecture, which also changes input representation and parameter count (7,850 to 203,530).	Architecture at an approximately equal parameter count (24,274 to 25,337).
Models	Linear, MLP, CNN, LSTM, Transformer (Section 2.3).	MLP with 31 hidden units (24,655); CNN with a 13-unit dense layer (25,337); LSTM unchanged (24,714); Transformer with width 36 and feed-forward 72 (24,274).
Fixed factors	Data split, normalization, optimizer, learning rate, batch size, 16-epoch budget, gradient clipping, checkpoint rule, seeds 42–46, hardware and thread count.	
Metrics	Primary: test macro-F1 of validation-selected checkpoints. Secondary: test accuracy, parameters, training time and inference time per image.	
Conclusion criterion	A difference between two models is treated as supported only if the gap between their mean test macro-F1 is larger than the larger of their two seed standard deviations.	

Table 10: Design of the two experiments.

2.7 Training and validation loop

The same epoch function receives an optimizer for training and no optimizer for validation. This controls both model mode and gradient recording. The following excerpt from `train.py` shows the update path.

Training/validation update path

```
training = optimizer is not None
model.train(training)
total_loss = 0.0
labels, predictions = [], []
with torch.set_grad_enabled(training):
    for images, targets in loader:
        images, targets = images.to(DEVICE), targets.to(DEVICE)
        if training:
            optimizer.zero_grad(set_to_none=True)
        logits = model(images)
        loss = nn.functional.cross_entropy(logits, targets)
        if not torch.isfinite(loss):
            raise FloatingPointError("non-finite loss")
        if training:
            loss.backward()
            nn.utils.clip_grad_norm_(model.parameters(), CLIP_NORM, error_if_nonfinite=True)
            optimizer.step()
```

Epoch loss weights each batch by its image count, including the smaller last batch. Accuracy and macro-F1 aggregate all predictions. Validation runs after every training epoch. The highest validation macro-F1 selects the saved weights; an exact tie retains the earlier epoch.

2.8 Reproduction and Recorded environment

The runs use two recorded environments. Both use PyTorch 2.14.0, scikit-learn 1.9.1, four CPU threads and float32 computation. The 25 main runs and the 15 ablation runs use Python 3.11.16, NumPy 2.4.6 and macOS 15.6.1 arm64 (commits `3c06e59` and `92f65c7`). The 45 extension runs and `03_analysis.ipynb` use Python 3.12.13, NumPy 2.5.3 and macOS 26.7.1 arm64 (commit `4f6a1f5` with the extension code, committed later as `7bde7de`). The code was tested with NumPy 2.4.6 and 2.5.3; `requirements.txt` installs 2.5.3.

Run the following commands from the `dl-co3133` repository root.

```
python3.11 -m venv .venv
.venv/bin/python -m pip install -r requirements.txt
.venv/bin/python Assignment1/download.py
cd Assignment1
../.venv/bin/python train.py
../.venv/bin/python evaluate.py
../.venv/bin/python train_budget.py
```

`download.py` fetches the four IDX files. `train.py` trains the five models for seeds 42–46 into `Assignment1/outputs/`, one folder per model and seed. The same loop also trains the nine extension models of Sections 3.10.3–3.10.5 into the same folder. `evaluate.py` computes validation and test metrics and inference time for every run and writes `metrics.json`. `train_budget.py` runs Experiment 2 into `Assignment1/outputs_budget/`. Each `best.pt` holds the selected weights, split indices and normalization statistics; `checkpoint.pt` holds the final-epoch weights. Results are analyzed in `02_results.ipynb`. `03_analysis.ipynb` loads `best.pt` and `test_predictions.npz` of the saved runs, aggregates per-class and error results over five seeds and analyses the extension runs from their saved checkpoints. It trains nothing and writes one file, `Assignment1/outputs/robustness_scores.csv`, so later runs reuse the robustness scores. It stops if any clean test macro-F1 differs from `metrics.json`; all 70 runs passed. Timings are compared only within one environment; the extension runs also shared the CPU, so their times are not compared.

3 Implementation results

All five models follow the protocol of Section 2.6, and every result uses the weights selected by validation macro-F1. Sections 3.1–3.2 report the predefined primary run (seed 42). Section 3.3 repeats the comparison over seeds 42–46 and applies the conclusion criterion of Section 2.6. Confusion matrices, per-class F1, error counts, the ablation, robustness and interpretability statistics are aggregated over all five seeds. Learning curves, galleries and example maps show seed 42.

3.1 Validation comparison

Accuracy and macro-F1 use the same selected checkpoint. The accuracy gap is training minus validation accuracy in percentage points, measured with those fixed weights.

Model	Accuracy (0–1)	Macro-F1	Selected epoch	Epochs run	Train–val gap (pp)
Linear	0.8653	0.8658	12	16	0.29
MLP	0.9073	0.9075	14	16	3.94
CNN	0.9233	0.9237	13	16	2.59
LSTM	0.8965	0.8970	13	16	0.59
Transformer	0.8842	0.8833	16	16	0.77

Table 11: Validation results of the selected checkpoints (seed 42).

CNN has the highest validation macro-F1, 0.9237, and is selected at epoch 13. Its validation accuracy is 92.33%, compared with 90.73% for MLP, 89.65% for LSTM, 88.42% for Transformer and 86.53% for Linear. MLP improves validation accuracy by 4.20 percentage points over Linear ($90.73 - 86.53$).

The train–val gap is largest for MLP (3.94 points) and CNN (2.59). LSTM and Transformer have smaller gaps (0.59 and 0.77) but also lower validation scores, so a small gap alone does not identify the strongest model.

Model	Parameters	Training (s)	Validation ms per image	Test ms per image
Linear	7,850	5.33	0.0013	0.0012
MLP	203,530	9.40	0.0019	0.0021
CNN	105,866	111.56	0.0276	0.0268
LSTM	24,714	46.22	0.0244	0.0223
Transformer	19,530	103.29	0.0553	0.0550

Table 12: Parameter count, training time and batched CPU inference time (seed 42).

CNN has about half as many parameters as MLP (105,866 versus 203,530) but is about 13 times slower at test inference (0.0268 versus 0.0021 ms per image), because shared convolution weights are applied at every image position. LSTM and Transformer have fewer parameters still, yet are also slower than MLP. Parameter count therefore does not measure computation.

Timing definitions are given in Section 2.5. The main runs share one recorded environment (Section 2.8), so timings are comparable only within this set of runs.

3.2 Held-out test results

The official test set evaluates the validation-selected checkpoints.

Model	Loss	Accuracy (0–1)	Macro-F1	Inference (ms/image)
Linear	0.4438	0.8403	0.8408	0.0012
MLP	0.3463	0.8897	0.8899	0.0021
CNN	0.2621	0.9084	0.9088	0.0268
LSTM	0.3384	0.8776	0.8781	0.0223
Transformer	0.3571	0.8688	0.8673	0.0550

Table 13: Test results of the validation-selected checkpoints (seed 42).

The validation-selected CNN reaches 90.84% test accuracy and 0.9088 macro-F1, 1.49 points below its validation accuracy (92.33 − 90.84). The other test accuracies are MLP 88.97%, LSTM 87.76%, Transformer 86.88% and Linear 84.03%.

MLP improves over Linear by 4.94 points (88.97 − 84.03), and CNN adds 1.87 points over MLP (90.84 − 88.97). The validation and test rankings agree: CNN > MLP > LSTM > Transformer > Linear. Section 3.3 checks this ranking for other training seeds.

3.3 Training-seed variability

Each model was trained with seeds 42–46 on the same split. The seed changes weight initialization and batch order only, so the standard deviation measures training variability on this split, not uncertainty across datasets.

Model	Validation macro-F1 (%)	Test accuracy (%)	Test macro-F1 (%)	Test macro-F1 median (%)	Mean training time (s)
Linear	86.51 ± 0.07	84.17 ± 0.08	84.15 ± 0.11	84.11	4.15
MLP	90.54 ± 0.17	88.77 ± 0.26	88.77 ± 0.23	88.83	7.77
CNN	92.56 ± 0.21	91.26 ± 0.28	91.27 ± 0.27	91.37	105.36
LSTM	89.80 ± 0.11	88.11 ± 0.25	88.10 ± 0.21	88.11	46.37
Transformer	88.48 ± 0.33	87.15 ± 0.39	87.06 ± 0.36	87.00	102.85

Table 14: Results over five training seeds (42–46). Performance metrics show mean ± sample standard deviation; the median column is the middle of the five test macro-F1 values; training time shows the mean only.

Comparison	Mean difference (pp)	Minimum (pp)	Maximum (pp)	First model better
MLP − Linear	4.62	4.31	4.92	5/5
CNN − MLP	2.50	1.89	2.86	5/5
LSTM − MLP	−0.66	−1.18	−0.14	0/5
CNN − LSTM	3.16	3.01	3.36	5/5
LSTM − Transformer	1.04	0.48	1.57	5/5

Table 15: Test macro-F1 differences paired by seed.

The largest standard deviation of test macro-F1 is 0.36 points (Transformer, Table 14). Every mean difference in Table 15 is larger than this in absolute value, and the ranking CNN > MLP > LSTM > Transformer > Linear is the same in all

five seeds. Under the criterion of Section 2.6, this ranking is therefore supported. Five seeds on one split are not used to claim statistical significance. Medians are within about 0.1 points of the means, so no single seed drives a mean.

3.4 Learning curves

Dashed lines show training metrics during updates; solid lines show validation after each epoch. Vertical lines mark the selected checkpoints. All models complete 16 epochs. Linear selects epoch 12: validation macro-F1 is 0.8658 there and 0.8577 at epoch 16, while validation loss rises from 0.3886 to 0.4003. MLP selects epoch 14; its training loss continues falling while validation loss rises from 0.2870 to 0.2942 by epoch 16. CNN selects epoch 13; by epoch 16 its validation loss rises from 0.2210 to 0.2284 and macro-F1 falls from 0.9237 to 0.9210. Later weights are therefore not necessarily the better generalizing weights.

LSTM reaches a validation macro-F1 of 0.8970 (rounded) at epochs 13 and 15, with epoch 13 retained as the selected checkpoint. Transformer selects epoch 16, the final epoch, so it may not have converged within the 16-epoch budget. For the other four models, `best.pt` and the final `checkpoint.pt` contain weights from different epochs.

Model	Seed 42	Seed 43	Seed 44	Seed 45	Seed 46
Linear	12	11	15	15	5
MLP	14	16	14	16	12
CNN	13	13	15	16	16
LSTM	13	16	15	13	14
Transformer	16	16	15	16	16

Table 16: Selected epoch for every model and seed (16 is the last epoch).

Transformer selects the last epoch in four of five seeds, so its possible under-training is not specific to seed 42. The other models select an earlier epoch in at least three seeds.

An earlier set of seed-42 runs (21 September) had been trained with uncommitted code changes in a different software environment; this was found from the environment and commit fields saved in each `checkpoint_info.json`. The reported results use only the later runs and their two recorded environments (Section 2.8). Cross-entropy requires int64 labels of shape $[B]$, so the dataset converts labels once when it is built. Non-finite losses or gradients stop training with an error. `evaluate.py` skips folders that already contain `metrics.json`, so that file must be deleted before re-evaluating a changed run.

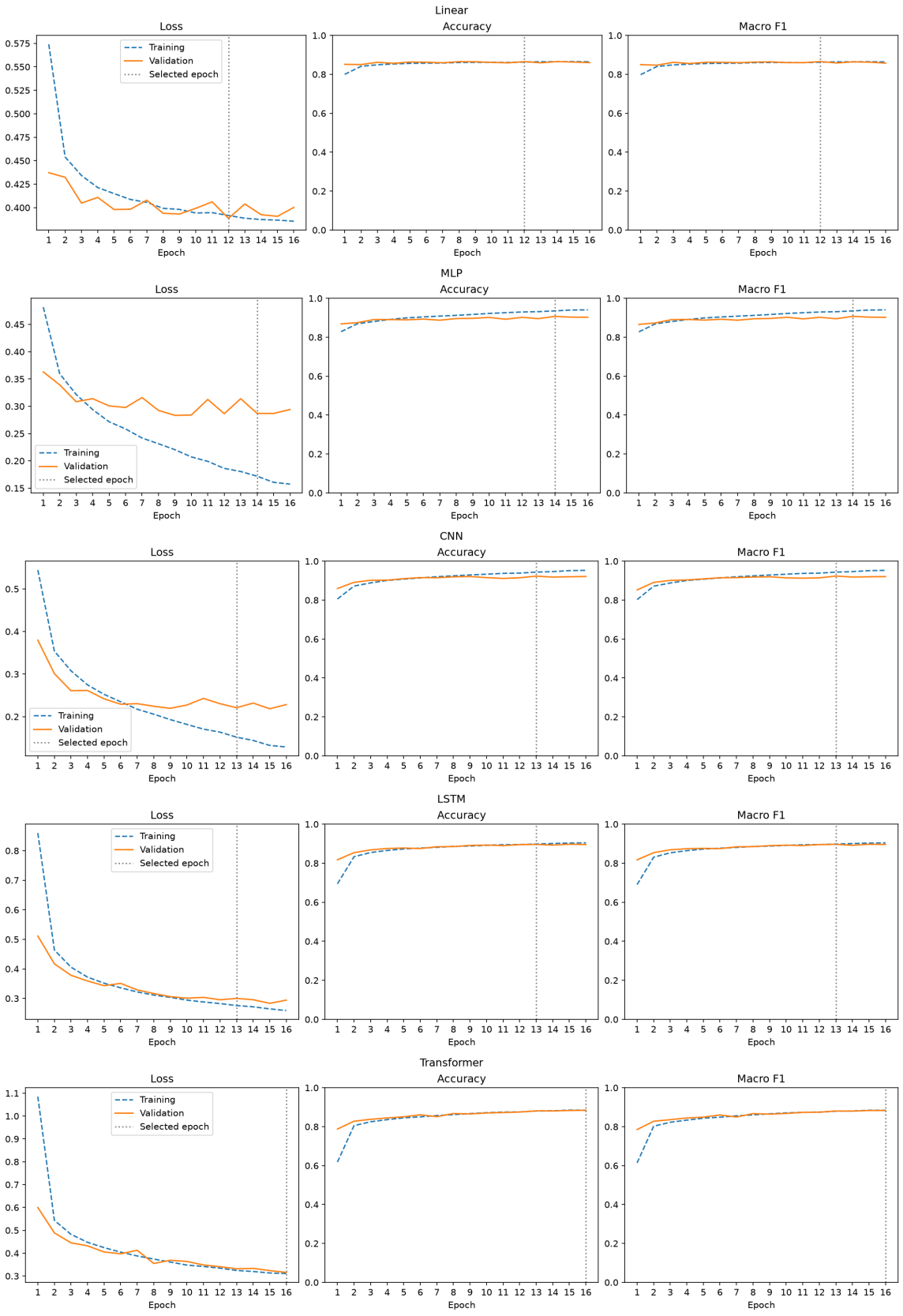
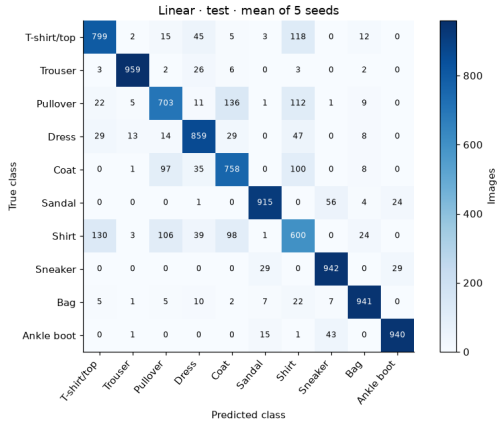


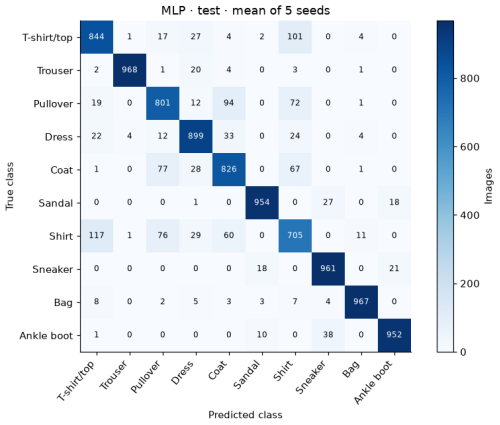
Figure 19: Learning curves of Linear, MLP, CNN, LSTM and Transformer (top to bottom; seed 42).

3.5 Test confusion matrices

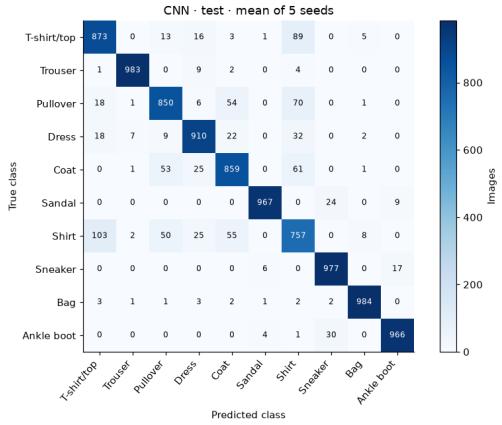
Rows are true classes; columns are predictions. Each cell is the mean count over seeds 42–46, rounded to whole images. Compare clothing confusions with the visual similarities observed in EDA.



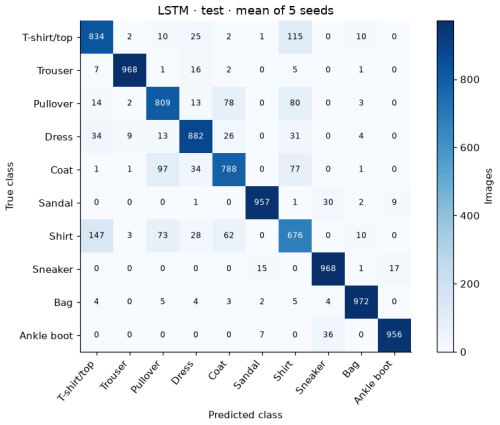
(a) Linear



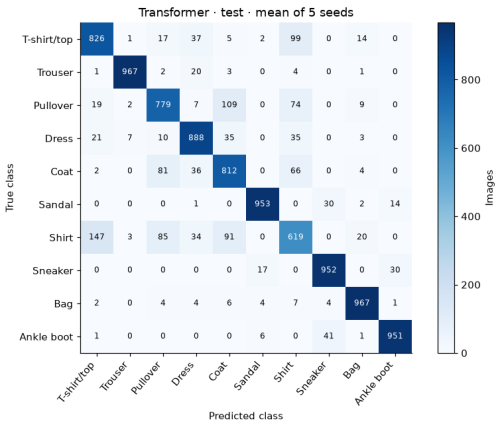
(b) MLP



(c) CNN



(d) LSTM



(e) Transformer

Figure 20: Test confusion matrices, mean of seeds 42–46. Exact means and seed ranges for the most frequent errors are listed in Table 18.

All five models make most of their mistakes among T-shirt/top, Shirt, Pullover and Coat, consistent with the EDA hypothesis about similar upper-body silhouettes. CNN reduces mean Pullover → Coat errors from Linear's 136.0 to 54.0, but its largest error is Shirt → T-shirt/top (102.8). Better overall performance does not improve every class pair equally: LSTM and Transformer make more Shirt → T-shirt/top errors (147.0 and 146.8) than Linear (130.4).

Each true class has 1,000 test images, so a count divided by ten is the percentage of that class.

Class	**Linear**	**MLP**	**CNN**	**LSTM**	**Transformer**
T-shirt/top	0.804 ± 0.006	0.838 ± 0.006	0.866 ± 0.003	0.817 ± 0.006	0.819 ± 0.007
Trouser	0.967 ± 0.002	0.981 ± 0.002	0.986 ± 0.002	0.975 ± 0.002	0.977 ± 0.001
Pullover	0.724 ± 0.009	0.806 ± 0.007	0.860 ± 0.007	0.805 ± 0.010	0.788 ± 0.014
Dress	0.848 ± 0.005	0.890 ± 0.005	0.912 ± 0.004	0.880 ± 0.005	0.876 ± 0.004
Coat	0.746 ± 0.005	0.815 ± 0.006	0.860 ± 0.005	0.804 ± 0.007	0.787 ± 0.013
Sandal	0.928 ± 0.004	0.961 ± 0.006	0.977 ± 0.005	0.965 ± 0.005	0.961 ± 0.003
Shirt	0.599 ± 0.009	0.712 ± 0.008	0.751 ± 0.007	0.679 ± 0.008	0.650 ± 0.013
Sneaker	0.919 ± 0.004	0.947 ± 0.006	0.961 ± 0.006	0.950 ± 0.005	0.940 ± 0.003
Bag	0.937 ± 0.002	0.972 ± 0.002	0.983 ± 0.001	0.970 ± 0.002	0.956 ± 0.004
Ankle boot	0.943 ± 0.003	0.956 ± 0.004	0.970 ± 0.004	0.964 ± 0.003	0.953 ± 0.003

Table 17: Per-class test F1, mean $\pm$ sample standard deviation over seeds 42–46.

Shirt is the weakest class for every model in every seed: mean F1 ranges from 0.599 for Linear to 0.751 for CNN. CNN has the highest mean F1 for all ten classes. Its Trouser F1 is 0.986 and Bag F1 is 0.983, so difficulty varies strongly despite equal class counts. Macro-F1 keeps the weak Shirt class visible in the overall score.

Model	True	Predicted	Mean count ± SD	Range	Class error rate	In seed's top five
Linear	Pullover	Coat	136.0 ± 24.2	100–166	0.136	4/5
Linear	Shirt	T-shirt/top	130.4 ± 18.1	115–160	0.130	5/5
Linear	T-shirt/top	Shirt	118.2 ± 22.0	86–147	0.118	4/5
Linear	Pullover	Shirt	111.6 ± 23.0	88–145	0.112	3/5
Linear	Shirt	Pullover	105.6 ± 16.5	90–126	0.106	3/5
MLP	Shirt	T-shirt/top	116.6 ± 32.6	84–167	0.117	5/5
MLP	T-shirt/top	Shirt	101.2 ± 38.0	59–143	0.101	4/5
MLP	Pullover	Coat	94.2 ± 22.7	70–124	0.094	4/5
MLP	Coat	Pullover	76.8 ± 21.1	50–106	0.077	3/5
MLP	Shirt	Pullover	76.2 ± 6.4	67–82	0.076	3/5
CNN	Shirt	T-shirt/top	102.8 ± 15.6	82–126	0.103	5/5
CNN	T-shirt/top	Shirt	88.6 ± 9.5	73–98	0.089	5/5
CNN	Pullover	Shirt	69.8 ± 11.9	53–83	0.070	4/5
CNN	Coat	Shirt	61.4 ± 18.2	45–91	0.061	2/5
CNN	Shirt	Coat	55.4 ± 15.0	31–72	0.055	3/5
LSTM	Shirt	T-shirt/top	147.0 ± 17.0	124–168	0.147	5/5
LSTM	T-shirt/top	Shirt	115.2 ± 25.4	88–152	0.115	5/5
LSTM	Coat	Pullover	96.8 ± 22.0	67–117	0.097	4/5
LSTM	Pullover	Shirt	80.4 ± 15.9	64–104	0.080	3/5
LSTM	Pullover	Coat	77.8 ± 22.3	60–116	0.078	1/5
Transformer	Shirt	T-shirt/top	146.8 ± 37.5	108–192	0.147	5/5
Transformer	Pullover	Coat	109.2 ± 22.4	76–135	0.109	4/5
Transformer	T-shirt/top	Shirt	98.6 ± 38.1	59–142	0.099	4/5
Transformer	Shirt	Coat	90.8 ± 12.8	72–105	0.091	5/5
Transformer	Shirt	Pullover	85.4 ± 22.6	60–118	0.085	3/5

Table 18: Five most frequent directional test errors per model, ranked by mean count over seeds 42–46. "In seed's top five" counts the seeds in which the pair is among that seed's five largest errors.

All 25 listed directional errors are between the four upper-body classes. Shirt → T-shirt/top is in the top five of every seed for every model. Single directional counts vary much more between seeds than total errors do: MLP's T-shirt/top → Shirt count ranges from 59 to 143. The direction of an error therefore depends on the seed; seed 42 alone (Linear 147 versus 121, MLP 143 versus 84) suggested a T-shirt/top → Shirt bias that the five-seed means reverse.

3.6 Inspect predictions

For CNN and Transformer, the figures show four correct predictions in test-file order, the four lowest-confidence correct predictions, and the first four incorrect predictions in test-file order. Confidence is the largest softmax probability; it is not a calibrated guarantee. IDs refer to the official test image file. The same views for Linear, MLP and LSTM are in `02_results.ipynb`.

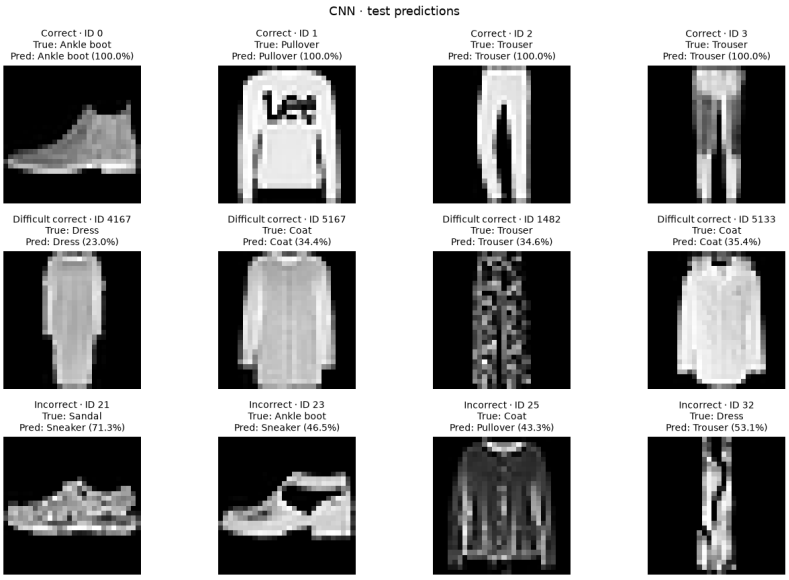


Figure 21: CNN — Inspect predictions (seed 42).

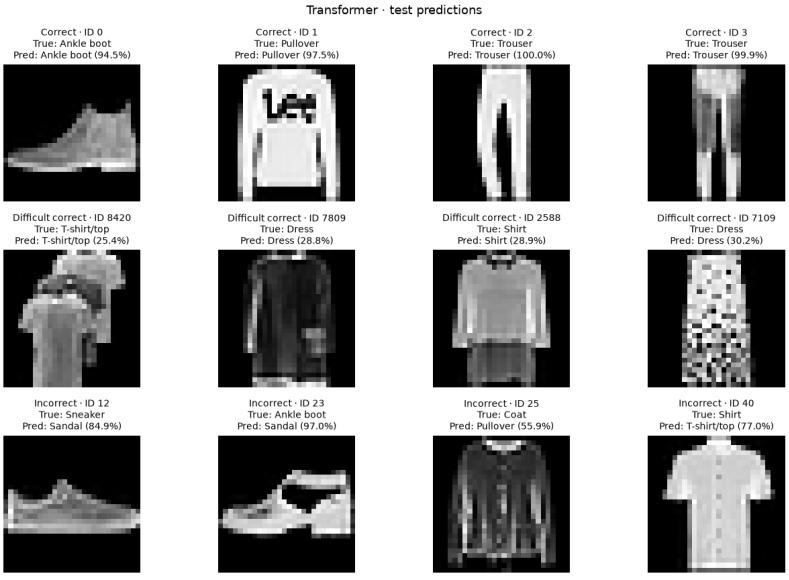


Figure 22: Transformer — Inspect predictions (seed 42).

CNN correctly recognizes the first boot, printed pullover and two trousers. Its low-confidence correct cases include Dress #4167 at 23.0% and Coat #5167 at 34.4%, both with plain, elongated outlines shared by several classes.

Transformer misclassifies Sneaker #12 as Sandal at 84.9% and Ankle boot #23 as Sandal at 97.0%; the latter has a heel and a large dark opening that resemble a sandal. CNN also misses #23 (as Sneaker, 46.5%), and labels Sandal #21 as

Sneaker (71.3%), dark Coat #25 as Pullover (43.3%) and narrow Dress #32 as Trouser (53.1%). These examples show visual ambiguity; they do not establish that the labels are wrong. Transformer's lowest-confidence correct example is T-shirt/top #8420 at 25.4%, an image with overlapping garments. Softmax confidence can be high on mistakes and low on correct decisions. The errors shown are the first four in file order, not a representative sample.

Across seeds 42–46, Ankle boot #23 is misclassified by every model in all five seeds, so it is a stable hard image. The other CNN errors shown are less stable: #25 fails in 4/5 seeds, #21 and #32 in 2/5. Galleries from one seed can therefore show errors that most seeds do not make.

3.7 Error analysis

Test errors from seeds 42–46 are grouped by the classes involved: *upper-body* errors have both the true and the predicted class in {T-shirt/top, Pullover, Coat, Shirt}; *footwear* errors have both in {Sandal, Sneaker, Ankle boot}; all remaining errors are *other*. *Confident* errors have softmax confidence of at least 0.9 and overlap the three groups.

Model	**All errors**	**Upper-body**	**Footwear**	**Other**	**Confident** ($p \geq 0.9$)
Linear	1,583 ± 8	939 ± 21 (59.3%)	196 ± 8 (12.4%)	449 ± 21 (28.3%)	158 ± 9 (10.0%)
MLP	1,123 ± 26	706 ± 15 (62.8%)	131 ± 13 (11.6%)	287 ± 14 (25.5%)	259 ± 43 (23.1%)
CNN	874 ± 28	568 ± 21 (65.0%)	89 ± 14 (10.2%)	217 ± 10 (24.8%)	220 ± 32 (25.1%)
LSTM	1,189 ± 25	757 ± 20 (63.6%)	114 ± 14 (9.6%)	318 ± 5 (26.8%)	158 ± 25 (13.3%)
Transformer	1,285 ± 39	794 ± 29 (61.7%)	139 ± 7 (10.8%)	353 ± 20 (27.5%)	134 ± 12 (10.4%)

Table 19: Test errors by group, mean ± sample standard deviation over seeds 42–46. Percentages are shares of each model's mean errors.

Upper-body confusions are 59–65% of errors for every model and footwear confusions another 10–12%, except LSTM (9.6%). CNN reduces mean upper-body errors from 939 (Linear) to 568, but their share does not fall. Confident errors are 25.1% of CNN errors and 23.1% of MLP errors, compared with 10.0–13.3% for Linear, LSTM and Transformer; the stronger models are wrong less often but more confidently.

Five seeds also show which errors are stable. Table 20 counts test images misclassified in at least one seed and in all five seeds.

Model	**Mean errors per seed**	**Wrong in ≥1 seed**	**Wrong in all 5 seeds**	**All-5 errors that are upper-body images**
Linear	1,583	2,007	1,175 (74%)	860
MLP	1,123	1,836	559 (50%)	421
CNN	874	1,642	318 (36%)	242
LSTM	1,189	1,962	595 (50%)	450
Transformer	1,285	2,260	512 (40%)	391

Table 20: Stability of test errors over seeds 42–46. Percentages compare all-five-seed errors with the mean errors per seed.

Linear has a convex loss, which probably explains why most of its errors repeat. For CNN only 318 of about 874 errors per seed repeat in all seeds; the rest depend on initialization and batch order. 146 test images are misclassified in all 25 runs (5 models × 5 seeds); 48 are Shirts and 35 are Pullovers.

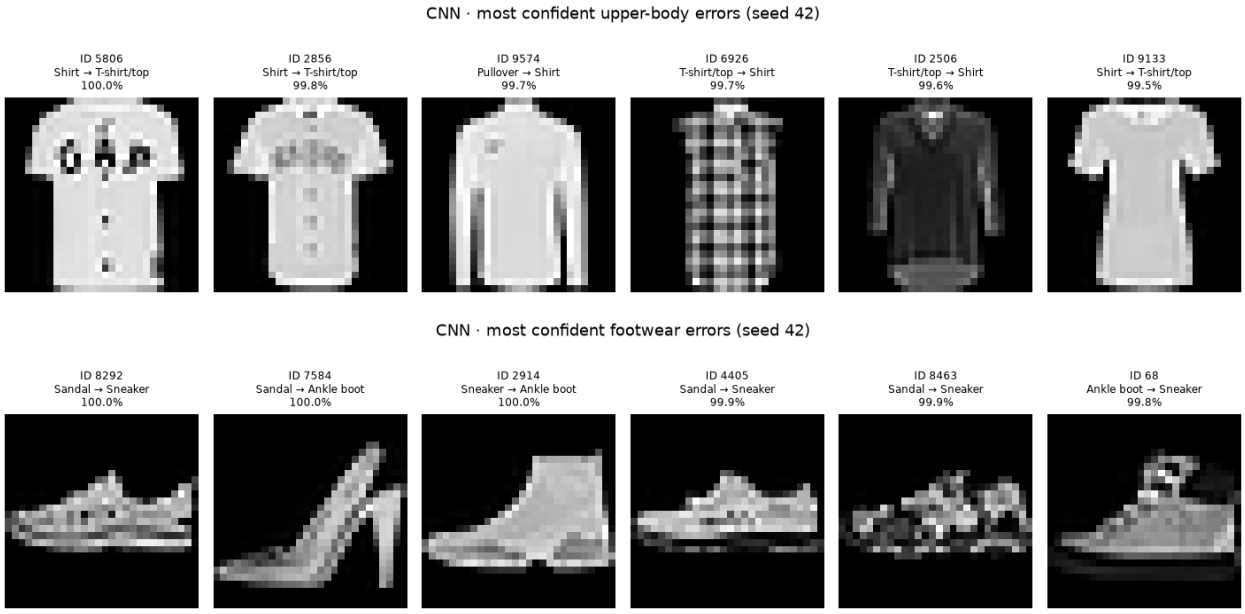


Figure 23: CNN's six most confident upper-body errors (top) and footwear errors (bottom), seed 42.

Many confident errors are visually ambiguous. Shirts #5806 and #2856 have short sleeves and look like T-shirts; Sandal #8292 resembles a sneaker and Sneaker #2914 an ankle boot. Some images look closer to the predicted class than to their label; we did not check whether this is label noise. All six upper-body errors in Figure 23 are misclassified by all five models in all five seeds, so they are properties of the images, not of one CNN run. Table 21 summarizes the error groups for CNN.

Group	**Description**	**CNN errors**	**Examples (test ID)**	**Hypothesized cause**	**Improvement idea**	**Why this may help**
Upper-body garments	T-shirt/top, Pullover, Coat, Shirt confused	65.0% (568)	#5806, #2856, #6926	Similar outlines; collars and buttons cover only a few pixels	Small image shifts as augmentation; a wider CNN	Augmentation can make the model less sensitive to small position changes, while a wider CNN may learn more detailed features that separate similar garments.
Footwear	Sandal, Sneaker, Ankle boot confused	10.2% (89)	#8292, #7584, #2914	Similar low shapes; straps and ankle height are thin details; some images may have ambiguous labels (not verified)	Small image shifts as augmentation; a wider CNN	These methods may help the model capture small shape differences such as straps, soles and ankle height more reliably.
Other	Remaining errors, largest cells involve Dress, e.g. Dress → Shirt 32.2 and Shirt → Dress 25.4 (Figure 20c)	24.8% (217)	#32 (Dress → Trouser)	Long or narrow dresses resemble long tops, coats and trousers	Same as upper-body garments	These errors also come from similar silhouettes.
Confident errors (overlap the rows above)	Wrong with confidence ≥ 0.9	25.1% (220)	#5806, #7584	High-confidence mistakes occur; calibration was not measured	Temperature scaling fitted on validation	Temperature scaling can make confidence better match actual correctness. It does not change which class is predicted, so it does not improve accuracy.

Table 21: Error analysis for CNN. Shares and counts are means over seeds 42–46; example IDs come from seed 42.

3.8 Ablation: matched parameter budget

Experiment 2 of Section 2.6 asks whether CNN leads because of its spatial design or because it has more parameters than LSTM and Transformer. All four models below have about 25k parameters and use seeds 42–46.

Model (≈25k)	Parameters	Validation macro-F1 (%)	Test macro-F1 (%)	Test ms per image	Original-model test macro-F1 (%)
MLP, 31 hidden units	24,655	88.91	87.01 ± 0.30	0.002	88.77 (203,530)
CNN, 13-unit dense layer	25,337	91.54	90.51 ± 0.34	0.029	91.27 (105,866)
LSTM (unchanged)	24,714	89.80	88.10 ± 0.21	0.022	88.10 (24,714)
Transformer, width 36	24,274	88.36	87.00 ± 0.54	0.071	87.06 (19,530)

Table 22: Matched-budget ablation over seeds 42–46. Values are means, with sample standard deviation shown for test macro-F1.

At about 25k parameters, CNN has the highest mean test macro-F1 (90.51), more than two points above every other model, while no standard deviation exceeds 0.54 (Table 22). Paired by seed, CNN's test macro-F1 is higher than that of every other budget model in 5/5 seeds, on average by 3.50 points over MLP, 2.40 over LSTM and 3.51 over Transformer; each gap is larger than the larger of the two seed standard deviations (Section 2.6). Validation macro-F1 agrees in 5/5 seeds (2.62, 1.73 and 3.18 points). CNN's lead in the main comparison is therefore not explained by its larger parameter count.

LSTM (88.10) is above the budget MLP (87.01) and Transformer (87.00). The budget MLP and Transformer have almost the same mean, so this experiment cannot separate them.

How size affects each model.

- **MLP drops the most.** From 203,530 to 24,655 parameters, test macro-F1 falls from 88.77% to 87.01% (-1.76 points).
- **CNN drops much less.** From 105,866 to 25,337 parameters, it falls from 91.27% to 90.51% (-0.76 points). The small CNN is still better than the full-size MLP.
- **Transformer does not improve with more parameters.** From 19,530 to 24,274 parameters, the difference is not supported (87.06% and 87.00%; seed standard deviations 0.36 and 0.54). This small test does not show what limits the Transformer; the 16-epoch budget is one possible reason (not tested).

Equal parameter counts do not mean equal computation: the budget CNN and Transformer remain slower per image than the budget MLP. Shrinking one layer is only one way to match a budget.

3.9 Results discussion

MLP improves mean test macro-F1 from 84.15% to 88.77% over Linear and is better in all five seeds (Table 15). Both models use flattened pixels, but MLP can combine them nonlinearly. They also differ in parameter count, so nonlinearity is not isolated as the only cause. The nonlinearity part of the Experiment 1 hypothesis (Section 2.6) is supported.

CNN has the highest mean test macro-F1, 91.27 $\pm$ 0.27%. It is better than MLP in all five seeds with about half the parameters, and it still leads when all models have about 25k parameters. LSTM is below the full MLP but above the MLP of the same size (88.10 versus 87.01, Table 22), so reading rows in order uses parameters more efficiently than flattening. Transformer does not beat the MLP of the same size within 16 epochs. The spatial-efficiency part of the hypothesis, tested by Experiments 1 and 2, is therefore supported for CNN, partly supported for LSTM and not supported for Transformer in this setting.

CNN is the most accurate model; MLP is much cheaper at inference (0.0021 versus 0.0268 ms per image, Table 13). LSTM (24,714 parameters) and Transformer (19,530) are much smaller than MLP (203,530) but slower at inference, because they repeat computation across rows or patches. Mean training time is 105.36 s for CNN and 102.85 s for Transformer versus 7.77 s for MLP (Table 14).

Shirt is the weakest class for every model and seed, and CNN still confuses Shirt and T-shirt/top in both directions (102.8 and 88.6 test images on average). High softmax confidence does not guarantee a correct label. These errors match the overlapping silhouettes found in EDA; 146 test images are wrong in all 25 runs.

3.10 Post-results analysis

This section reuses the 25 main runs and adds 45 extension runs trained with the same protocol. Robustness and interpretability use the saved checkpoints; GRU, the representation ablation and shift + noise augmentation are new models.

3.10.1 Robustness evaluation

This optional extension asks whether the inductive bias of each representation also changes how a model fails when test images change. No model is retrained; the 25 selected checkpoints of Experiment 1 are evaluated on corrupted copies of the test set.

Item	Setting
Hypothesis	CNN tolerates small shifts better than flattened-input models; LSTM, which reads rows in order, tolerates vertical shifts better than horizontal shifts.
Changed factor	The corruption and its level (Table 24).
Fixed factors	Checkpoints, training-split normalization, test labels. Noise and square positions use fixed random seeds, so every model and seed sees the same corrupted images.
Metrics	Test macro-F1, mean $\pm$ sample standard deviation over seeds 42–46; drop from the clean score paired by seed. Shift scores average the four directions.
Criterion	Same as Section 2.6.

Table 23: Design of the robustness evaluation.

A *corruption* is a fixed change to the pixels of a test image before the model sees it. Let $x(i, j) \in [0, 1]$ be the pixel in row i and column j (raw value divided by 255; background is 0). The corrupted image x' is then normalized with the training mean and standard deviation, exactly like a clean image.

Corruption	Rule for each pixel	Levels	What it imitates
Noise	$x' = \min(1, \max(0,\ x + \varepsilon))$, with $\varepsilon \sim \mathcal{N}(0, \sigma^2)$ drawn independently for each pixel	$\sigma = 0.05, 0.10, 0.20$ (13, 26, 51 of 255 grey levels)	Sensor noise; the black background becomes grey speckle
Shift	$x'(i, j) = x(i - d_y,\ j - d_x)$; pixels that enter from outside the image are 0	$d = 1, 2, 3$ pixels, in one of four directions: up, down, left, right	A garment that is not centred
Occlusion	$x' = 0$ inside one $k \times k$ square at a random position; $x' = x$ elsewhere	$k = 8$ (8% of the image), $k = 12$ (18%)	Part of the garment hidden; the square is black, like the background
Contrast	$x' = c\,x$	$c = 0.75, 0.5$	A darker photo; the garment fades and the background stays 0

Table 24: Definition of the corruptions. Each level is one test set of 10,000 corrupted images; every model and seed sees the same corrupted images.

Why noise? In the EDA, 50.2% of all training pixels are exactly 0 (Figure 5), so in training the background is always black. Noise turns it into grey speckle that no model has seen. It also tests the inductive bias: a 3×3 filter or a 4×4 patch sees 9 or 16 pixels, while a dense layer sums all 784.

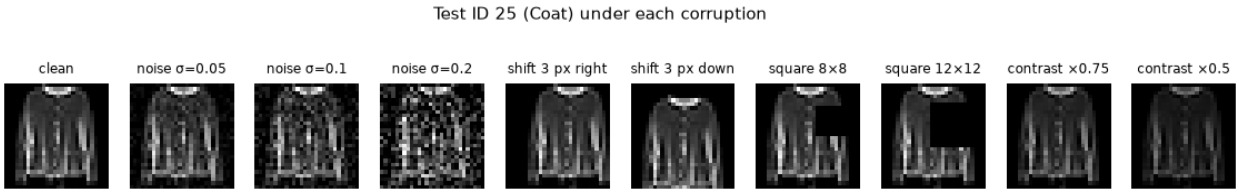


Figure 24: Test image #25 (Coat) under each corruption. The shifts shown are 3 pixels right and 3 pixels down.

Corruption	Linear	MLP	CNN	LSTM	Transformer
Clean	84.15 ± 0.11	88.77 ± 0.23	**91.27** ± 0.27	88.10 ± 0.21	87.06 ± 0.36
Noise $\sigma = 0.05$	83.76 ± 0.09	88.44 ± 0.23	**89.58** ± 0.64	87.79 ± 0.24	86.07 ± 0.28
Noise $\sigma = 0.10$	82.85 ± 0.13	**87.76** ± 0.21	84.47 ± 1.37	86.62 ± 0.44	82.41 ± 0.59
Noise $\sigma = 0.20$	80.22 ± 0.64	**85.21** ± 0.28	65.89 ± 3.66	77.28 ± 2.98	59.88 ± 4.24
Shift 1 px	76.00 ± 0.23	81.96 ± 0.23	**88.56** ± 0.18	82.92 ± 0.78	81.38 ± 1.24
Shift 2 px	59.79 ± 0.55	65.06 ± 0.98	**81.08** ± 0.84	64.64 ± 1.10	68.52 ± 2.34
Shift 3 px	42.85 ± 0.82	47.41 ± 1.62	**67.38** ± 1.76	51.95 ± 1.32	55.51 ± 2.11
Occlusion 8×8	76.91 ± 0.22	79.90 ± 0.46	**83.45** ± 0.78	77.23 ± 0.63	73.17 ± 2.01
Occlusion 12×12	66.19 ± 0.31	67.20 ± 1.29	**72.85** ± 1.75	62.70 ± 1.44	51.27 ± 4.01
Contrast $\times 0.75$	83.05 ± 0.37	87.93 ± 0.33	**90.38** ± 0.30	86.61 ± 0.38	86.62 ± 0.55
Contrast $\times 0.5$	73.37 ± 1.04	81.93 ± 1.63	**86.66** ± 1.17	80.44 ± 0.89	83.44 ± 0.41

Table 25: Test macro-F1 (%) under corruption, mean ± sample standard deviation over seeds 42–46. Bold marks the highest mean; it is also the highest model in all five seeds for every row.

CNN is the most robust model for shifts, occlusion and reduced contrast. Noise reverses the ranking: from $\sigma = 0.10$, MLP is best in all five seeds (87.76 versus 84.47 for CNN, a gap larger than either standard deviation).

Drop (pp)	Linear	MLP	CNN	LSTM	Transformer
Noise $\sigma = 0.20$	3.92	3.56	25.38	10.82	27.18
Shift 2 px	24.35	23.71	10.18	23.46	18.54
Occlusion 12×12	17.95	21.56	18.41	25.41	35.79
Contrast $\times 0.5$	10.77	6.84	4.61	7.66	3.62

Table 26: Mean drop in test macro-F1 from the clean score, paired by seed.

- **Noise hurts the local models.** CNN and Transformer lose more than 25 points at $\sigma = 0.20$; Linear and MLP lose less than 4. A 3×3 filter or a 4×4 patch sees only a few pixels, so independent pixel noise changes its input strongly, whereas a dense layer averages over all 784 pixels. This explanation was not tested directly.
- **Shifts hurt the position-bound models.** A 2-pixel shift costs Linear, MLP and LSTM about 24 points but CNN only 10.18. Shared filters and max pooling may give CNN partial translation tolerance; Linear and MLP learn each pixel position separately, and LSTM reads fixed row vectors. The EDA mean image (Figure 6) shows where clothing usually sits; these models likely rely on that position. These explanations were not tested.
- **Transformer is the least stable.** It loses the most under occlusion (35.79 points) and has the largest seed standard deviation in 8 of 11 conditions (up to 4.24). Under halved contrast it has the smallest mean drop (3.62 points; CNN 4.61), but only in 3/5 seeds.

Shift	Linear	MLP	CNN	LSTM	Transformer
2 px horizontal	51.92	57.97	79.74	48.90	62.70
2 px vertical	67.66	72.15	82.42	80.39	74.34
3 px horizontal	29.18	34.97	58.86	27.93	44.11
3 px vertical	56.52	59.85	75.90	75.96	66.92

Table 27: Mean test macro-F1 (%) for horizontal (left, right) and vertical (up, down) shifts over seeds 42–46.

Table 27 confirms the LSTM hypothesis. A vertical shift moves the row sequence by a few time steps, and LSTM keeps 75.96% at 3 pixels, close to CNN (75.90%). A horizontal shift changes every 28-pixel input vector, and LSTM falls to 27.93%, the lowest of all models. At 2 and 3 pixels, every model loses more to horizontal shifts, but the gap is 48 points for LSTM and 17 points for CNN.

3.10.2 Interpretability analysis

This optional extension asks which image regions each model uses, and whether the answer is stable across seeds. Three methods run on the saved checkpoints without retraining.

Method	Setting
Occlusion sensitivity (all models)	300 test images (30 per class, sampled with seed 0). A 4×4 patch is set to background at stride 2 (169 positions); each pixel's value is the mean drop in true-class probability over the patches that cover it.
LSTM row probe	The trained classifier is applied to the hidden state $\mathbf{h}_t$ after every row t on the full test set. Nothing is retrained.
Transformer attention	Attention weights of both layers are recomputed from the encoder; the manual pass reproduces the logits of `forward()` within 10^{-5}. Statistics: mean attention distance on the 7×7 patch grid, entropy divided by $\log 49$ (1 means uniform), and attention rollout.

Table 28: Methods of the interpretability analysis. All statistics use seeds 42–46.

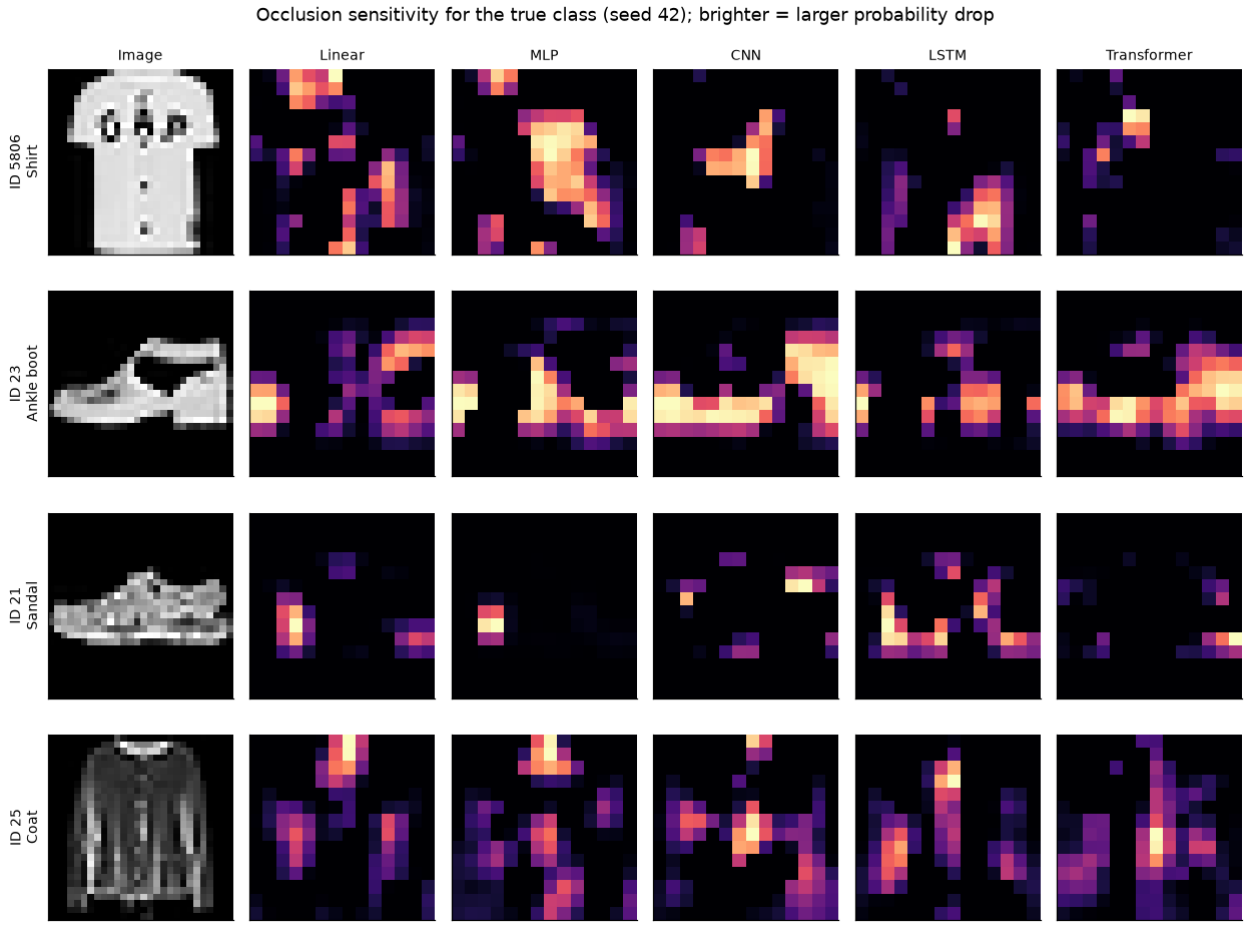


Figure 25: Occlusion sensitivity for the true class (seed 42); brighter means a larger probability drop, with one colour scale per map. The images are discussed in Sections 3.6 and 3.7.

For Shirt #5806, MLP uses the body and LSTM the lower rows, while CNN and Transformer react to a few small regions. For Ankle boot #23, which every run misclassifies, most models use the sole and the toe and heel edges; a sandal may also have these parts (not tested).

Model	Seed agreement r (images)	Pixels with half the evidence (%)	Upper-body: top third (%)	Upper-body: middle (%)	Upper-body: bottom third (%)
Linear	0.97 ± 0.02 (297)	7.3 ± 0.2	43.3 ± 1.0	30.6 ± 0.8	26.1 ± 0.5
MLP	0.76 ± 0.02 (247)	5.3 ± 0.1	42.6 ± 0.7	26.6 ± 0.8	30.8 ± 0.7
CNN	0.56 ± 0.02 (264)	5.8 ± 0.3	32.4 ± 2.1	36.4 ± 2.9	31.2 ± 3.8
LSTM	0.57 ± 0.02 (300)	6.0 ± 0.2	44.9 ± 2.3	22.8 ± 1.5	32.3 ± 2.5
Transformer	0.46 ± 0.02 (300)	8.3 ± 0.3	36.0 ± 1.3	36.3 ± 1.6	27.7 ± 1.0

Table 29: Occlusion statistics. Seed agreement is the mean Pearson correlation between maps of the same image from two seeds ($\pm$ standard deviation over the 10 seed pairs); images with a flat map in any seed (a fully confident model) are excluded. The half-evidence and band columns use only images whose map has positive evidence (at most 300 images, and at most 120 upper-body images for the bands; the retained counts were not recorded); the band columns show mean $\pm$ standard deviation over seeds of positive evidence shares; the top and bottom thirds are rows 1–9 and 20–28 (32% of the area each), and the middle is rows 10–19 (36%).

- **Explanations are seed-dependent.** Linear maps are almost identical across seeds ($r = 0.97$), but CNN, LSTM and Transformer maps agree only at $r = 0.46$–0.57. The models reach similar accuracy with different evidence, so a map from one seed should not be read as the model family's reasoning.
- **Upper-body evidence.** Linear, MLP and LSTM put 43–45% of their evidence for upper-body garments in the top third, which holds the collar, shoulders and sleeves. CNN and Transformer spread it closer to the area shares.
- **Concentration.** Half of the evidence lies in 5.3% of the pixels for MLP and 8.3% for Transformer, the most spread-out model.

Rows read	Accuracy (%)
7	20.78 ± 2.97
14	39.84 ± 2.09
18	54.91 ± 2.94
21	68.89 ± 2.17
24	82.59 ± 0.49
26	85.82 ± 0.31
28	88.11 ± 0.25

Class	Row where prediction becomes final
Bag	10.8 (8–13)
Dress	11.2 (7–13)
Trouser	12.4 (11–13)
Sandal	16.2 (10–20)
Ankle boot	17.6 (12–22)
Coat	18.3 (17–20)
Pullover	19.6 (18–21)
T-shirt/top	20.0 (19–21)
Sneaker	22.2 (21–23)
Shirt	23.4 (22–24)

Table 30: LSTM row probe over seeds 42–46. Left: test accuracy when the classifier reads $\mathbf{h}_t$ after t rows (mean $\pm$ standard deviation). Right: per class, the median row after which the prediction no longer changes, as the mean over seeds (range of the five seed medians).

LSTM decides late: accuracy is 39.84% after half of the rows and reaches the test accuracy (88.11%) only at row 28. Bag, Dress and Trouser are fixed after about 11–12 rows, probably because their outline is visible early (not tested); Shirt is fixed last (row 23.4), consistent with it being the weakest class. The classifier was trained only on $\mathbf{h}_{28}$, so early readings underestimate what the hidden state contains.

Layer	Mean attention distance (patches)	Uniform attention distance	Normalized entropy	Lowest head entropy
1	3.57 ± 0.04	3.61	0.870 ± 0.023	0.819
2	3.63 ± 0.05	3.61	0.488 ± 0.027	0.409

Table 31: Transformer attention on the full test set, mean $\pm$ standard deviation over seeds 42–46. Lowest head entropy is the mean over seeds of the lowest head; the uniform distance is a constant.

Layer 1 attention is close to uniform (entropy 0.870). Layer 2 is more selective (0.488), but its mean attention distance is not shorter than that of uniform attention (3.63 against 3.609 patches). Rollout gives the top two, middle three and bottom two patch rows 30.5%, 41.4% and 28.1% of the weight, close to their area shares (28.6%, 42.9%, 28.6%). These statistics do not show local attention like the fixed locality of CNN filters, and the mean-pooled output is close to an average of patch features. This may explain its weak occlusion robustness and its possible under-training within 16 epochs (not tested).

3.10.3 LSTM vs GRU

GRU removes the LSTM cell and keeps one hidden state [5]. A reset gate $\mathbf{g}_t$ controls how much of $\mathbf{h}_{t-1}$ enters the candidate, and an update gate $\mathbf{u}_t$ blends the old state with it:

$$\tilde{\mathbf{h}}_t = \tanh\big(W_n \mathbf{r}_t + \mathbf{g}_t \odot (U_n \mathbf{h}_{t-1})\big), \qquad \mathbf{h}_t = (1 - \mathbf{u}_t) \odot \tilde{\mathbf{h}}_t + \mathbf{u}_t \odot \mathbf{h}_{t-1}.$$

This is the PyTorch form, with biases omitted; the slides apply the reset gate before the weight matrix. When $\mathbf{u}_t \approx 1$, the state is copied forward, like the LSTM forget path. Only `nn.LSTM` changes to `nn.GRU`; rows, hidden size 64 and classifier are the same, so GRU has 18,698 parameters against 24,714.

Model	Clean	Noise $\sigma = 0.20$	Shift 2 px horizontal	Shift 2 px vertical	Square 12×12	Contrast $\times 0.5$
LSTM	88.10 ± 0.21	77.28 ± 2.98	48.90 ± 1.88	80.39 ± 0.46	62.70 ± 1.44	80.44 ± 0.89
GRU	88.88 ± 0.20	81.99 ± 0.91	53.00 ± 3.63	81.68 ± 0.26	63.50 ± 0.73	83.01 ± 0.93

Table 32: LSTM and GRU on the same row sequence: test macro-F1 (%), mean $\pm$ sample standard deviation over seeds 42–46.

GRU is better by 0.78 points in 5/5 seeds, larger than both standard deviations, so the difference is supported. It selects epoch 16 in 3/5 seeds and may still improve. GRU keeps LSTM's row asymmetry (53.00 horizontal, 81.68 vertical): the sequence direction, not the cell type, decides which shift hurts. The slides note that the winner is task-dependent [5]; on 28 short steps, the simpler gate may be easier to train in 16 epochs (not tested).

3.10.4 Representation ablation

The same model reads the same image cut in a different way. The encoder and training settings are fixed; the number of tokens, the token size and the input projection change with the cut, so TransformerRows has 19,242 parameters against 19,530.

Model (tokens × size)	**Clean**	**Paired gap, better seeds**	**Noise $\sigma = 0.20$**	**Shift 2 px horizontal**	**Shift 2 px vertical**	**Square 12×12**
LSTM, rows (28×28)	88.10 ± 0.21	–	77.28	48.90	80.39	62.70
LSTM, columns (28×28)	87.36 ± 0.23	-0.75, 0/5	72.59	80.74	71.68	54.72
Transformer, 4×4 patches (49×16)	87.06 ± 0.36	–	59.88	62.70	74.34	51.27
Transformer, 7×7 patches (16×49)	86.80 ± 0.43	-0.26, 2/5	69.89	54.96	72.31	64.98
Transformer, rows (28×28)	88.44 ± 0.31	$+1.38$, 5/5	78.62	53.67	79.54	62.25

Table 33: Representation ablation: test macro-F1 (%) over seeds 42–46. The paired gap compares each variant with the first row of its block.

- **Rows beat columns and small patches.** LSTM rows are better than columns in 5/5 seeds, and row tokens give the best Transformer (+1.38 points). A row is a full horizontal profile, such as sleeve width or a flat sole; many 4×4 patches are mostly background, since half of all pixels are 0. This is a likely cause, not tested.
- **The asymmetry flips with the reading direction.** Columns keep 80.74 under a horizontal shift and fall to 71.68 under a vertical shift, the opposite of rows. A shift along the sequence moves tokens in time; a shift across it changes every token.
- **Larger patches trade shift for stability.** For 7×7 patches, the clean macro-F1 difference is not supported (-0.26, 2/5 seeds), but they are more stable under noise (69.89 against 59.88) and occlusion (64.98 against 51.27).

3.10.5 Shift + noise augmentation

Section 3.10.1 found two weaknesses: Linear, MLP and LSTM fail under shifts, CNN and Transformer fail under noise. Each of the five models is retrained with one change: almost every *training* draw (49 of 50) changes the image. Validation, test and checkpoint selection use clean images. Table 34 separates this training change from the test-time noise.

	Test-time noise (Section 3.10.1)	**Training-time augmentation**
Shift	1, 2 or 3 pixels in one direction	Always: random $d_y, d_x \in \{-2, \ldots, 2\}$ pixels; 24 of 25 draws move the image
Noise rule	$x' = \min(1, \max(0, x + \varepsilon))$, $\varepsilon \sim \mathcal{N}(0, \sigma^2)$ per pixel	Same rule
Noise strength	$\sigma = 0.05, 0.10, 0.20$	$\sigma = 0.10$ only
Images with noise	Every test image	Each training draw with probability 0.5
Randomness	One fixed draw for all models	A new draw every epoch

Table 34: Test-time noise and training-time augmentation. The augmentation applies shift and noise together, so it is *shift + noise* augmentation, not training with noise alone.

Model	**Clean accuracy**	**Clean macro-F1**	$\sigma = 0.05$ **accuracy**	$\sigma = 0.10$ **accuracy**	$\sigma = 0.20$ **accuracy**	$\sigma = 0.20$ **macro-F1**
Linear	84.17 ± 0.08	84.15 ± 0.11	83.78 ± 0.09	82.89 ± 0.14	80.31 ± 0.61	80.22 ± 0.64
augmented	78.56 ± 0.34	78.49 ± 0.24	78.12 ± 0.40	77.38 ± 0.27	75.16 ± 0.28	74.92 ± 0.24
MLP	88.77 ± 0.26	88.77 ± 0.23	88.45 ± 0.28	87.77 ± 0.25	85.23 ± 0.31	85.21 ± 0.28
augmented	86.85 ± 0.15	86.78 ± 0.21	86.71 ± 0.16	86.31 ± 0.13	84.79 ± 0.09	84.70 ± 0.14
CNN	91.26 ± 0.28	91.27 ± 0.27	89.51 ± 0.61	84.51 ± 1.02	66.89 ± 3.68	65.89 ± 3.66
augmented	90.36 ± 0.25	90.35 ± 0.25	89.96 ± 0.20	88.89 ± 0.26	82.40 ± 1.06	82.70 ± 1.08
LSTM	88.11 ± 0.25	88.10 ± 0.21	87.79 ± 0.29	86.63 ± 0.52	77.32 ± 3.05	77.28 ± 2.98
augmented	86.52 ± 0.43	86.48 ± 0.46	86.39 ± 0.46	86.03 ± 0.48	83.39 ± 0.87	83.32 ± 0.88
Transformer	87.15 ± 0.39	87.06 ± 0.36	86.12 ± 0.28	82.37 ± 0.56	59.60 ± 4.19	59.88 ± 4.24
augmented	84.44 ± 0.57	84.03 ± 0.69	84.07 ± 0.47	83.12 ± 0.46	76.93 ± 1.73	76.91 ± 1.91

Table 35: Clean and noisy test images (%), mean $\pm$ sample standard deviation over seeds 42–46. Augmented models trained with $\sigma = 0.10$; $\sigma = 0.20$ is twice that level.

Model (base $\rightarrow$ augmented)	**Shift 2 px horizontal**	**Shift 2 px vertical**	**Square** 12×12	**Contrast** $\times 0.5$
Linear	51.92 $\rightarrow$ 75.88	67.66 $\rightarrow$ 74.86	66.19 $\rightarrow$ 51.33	73.37 $\rightarrow$ 57.77
MLP	57.97 $\rightarrow$ 85.76	72.15 $\rightarrow$ 84.77	67.20 $\rightarrow$ 61.56	81.93 $\rightarrow$ 78.91
CNN	79.74 $\rightarrow$ 89.51	82.42 $\rightarrow$ 89.05	72.85 $\rightarrow$ 70.48	86.66 $\rightarrow$ 83.59
LSTM	48.90 $\rightarrow$ 85.51	80.39 $\rightarrow$ 85.21	62.70 $\rightarrow$ 58.63	80.44 $\rightarrow$ 79.37
Transformer	62.70 $\rightarrow$ 83.09	74.34 $\rightarrow$ 82.27	51.27 $\rightarrow$ 48.74	83.44 $\rightarrow$ 79.30

Table 36: Test macro-F1 (%) under the other corruptions, mean over seeds 42–46.

- **Clean cost.** Augmentation lowers clean macro-F1 for every model in 5/5 seeds: CNN -0.92, LSTM -1.62, MLP -1.98, Transformer -3.03 and Linear -5.66 points. CNN already tolerates small shifts and pays least; Linear has one weight per pixel and cannot learn shift invariance.
- **Shift is repaired; noise is reduced.** At $\sigma = 0.20$, CNN macro-F1 rises from 65.89 to 82.70 and Transformer from 59.88 to 76.91. Under a horizontal 2-pixel shift, LSTM rises from 48.90 to 85.51 and its row asymmetry disappears. MLP loses a little (85.21 to 84.70); Linear loses 5.30 points (80.22 to 74.92), mostly its clean cost.
- **The gain does not transfer.** Occlusion and reduced contrast get worse on average for every model; Linear under contrast $\times 0.5$ falls from 73.37 to 57.77. Augmentation gives little or no robustness to changes not seen in training.
- **Caveats.** Shift and noise are applied together, so their effects cannot be separated. 11 of 25 runs select epoch 16, so part of the clean cost may come from the 16-epoch budget. `metrics.json` records `"augmentation": null` because `train.fit` writes a fixed value; the real augmentation is in `models/augmented.py`.

3.10.6 Summary

The post-results analysis shows that inductive bias also decides how a model fails. CNN's shared filters and pooling make it the most shift-tolerant model, but its local filters may make it fragile under noise, where MLP is best (not tested). LSTM's row sequence tolerates vertical shifts and fails under horizontal shifts (Table 27). Transformer attention is not shorter-range on average than uniform attention (Table 31), and it is the least robust model under occlusion. Clean macro-F1 alone does not show these differences. Rows beat columns and small patches for both sequence models (Table 33), GRU beats LSTM with fewer parameters (Table 32), and shift + noise augmentation removes most of the shift weakness and reduces the noise weakness at a clean cost (Section 3.10.5).

Results and EDA. The EDA predicted the hardest region: Pullover, Coat and Shirt have the closest class means and mix in t-SNE, and upper-body confusions are 59–65% of all errors. It did not predict the largest error: Shirt $\rightarrow$ T-shirt/top leads for four models, although that pair is only fourth closest (RMS 0.132). Two EDA facts fit the robustness failures: half of all pixels are 0, so noise changes the black background that training never varied, and the mean image suggests that garments sit in similar positions, which position-bound models may rely on. These links are explanations, not tested causes.

3.11 Limitations and conclusion

Limitations: All runs use one data split, five training seeds and a fixed 16-epoch budget with no tuning per architecture, so the scores are not each architecture's best possible result; Transformer still selects its final epoch in four of five seeds. Galleries and example maps come from seed 42 only. Robustness uses synthetic corruptions with one random draw each, and occlusion maps use at most 300 sampled images; neither covers real camera changes. Occlusion maps and the LSTM probe measure model sensitivity, not human-verified reasons. The augmentation applies shift and noise together, so their effects cannot be separated. Mechanisms marked as likely (row profiles, layer normalization, attention range) were not tested. Timings come from one CPU machine. The extension runs used a second software environment (Section 2.8), so a difference against a main run combines the changed factor with a software change; the extensions are exploratory, and earlier test results informed their design. Small, centred grayscale catalog images also limit transfer to real-world photographs.

Future work: Using only training and validation data, test a longer budget for Transformer, GRU and the augmented models, shift-only and noise-only augmentation, weight decay, and temperature scaling for calibration. These are proposals, not measured improvements.

Conclusion: In this setting, parameter count alone does not explain the performance differences; the image representation and its inductive bias matter. CNN has the highest mean test macro-F1 ($91.27 \pm 0.27\%$), and the ranking CNN > MLP > LSTM > Transformer > Linear is the same in all five seeds. MLP exceeds Linear by 4.62 points, but CNN needs about 13 times the MLP inference time. At about 25k parameters, CNN still leads every model in 5/5 seeds, and the 25,337-parameter CNN (90.51%) is better than the 203,530-parameter MLP (88.77%).

The errors concentrate on visually similar upper-body garments: Shirt is the weakest class in all 25 runs, and 146 test images are wrong in every run. The post-results analysis shows that each representation also fails in its own way. CNN is the most robust model to shifts, occlusion and reduced contrast, but pixel noise reverses the ranking: from $\sigma = 0.10$, MLP is best, and CNN and Transformer lose more than 25 points at $\sigma = 0.20$. The sequence direction decides which shift hurts a sequence model: LSTM keeps 75.96% under a 3-pixel vertical shift but only 27.93% under a horizontal shift. The interpretability measurements agree with these results at the level of sensitivity: explanations depend on the seed for every model except Linear, LSTM decides late, and Transformer attention is not shorter-range on average than uniform attention.

Within the sequence models, GRU is 0.78 points better than LSTM in 5/5 seeds with 24% fewer parameters, and row tokens make the Transformer 1.38 points better in 5/5 seeds. Shift + noise augmentation repairs most of the shift weakness and reduces the noise weakness of CNN, LSTM and Transformer, but it lowers clean macro-F1 in 5/5 seeds for every model, from -0.92 (CNN) to -5.66 points (Linear). Clean accuracy alone therefore does not show how a model fails; robustness needs its own test.

4 AI Usage Disclosure

The group takes full responsibility for all submitted code, numbers, figures, claims and text. AI tools were used as assistants for ideation, learning, drafting and language editing, not as substitutes for the group's work. No experimental result, metric, figure or data was generated by AI: every number in this report comes from the group's own runs (`outputs/`, `outputs_budget/`) and `02_results.ipynb`. Every member can explain any part of the submission. The dated usage log is in `AI_USAGE.md`.

Tool (provider)	**Version / access**	**Uses (below)**
Claude (Anthropic)	Claude Opus 5.5, claude.ai	1, 2, 3, 4
ChatGPT (OpenAI)	[model], chatgpt.com	[e.g. 1, 3, 4, 6]
NotebookLM (Google)	Web application	5

Table 37: AI tools used in Assignment 1.

1. **Ideation.** *Stage:* planning of each milestone. *Contribution:* high-level suggestions on how to organise the comparison and which questions each section should answer. *Prompt:* "What aspects should a comparative study of linear, MLP and CNN classifiers on Fashion-MNIST address?" *Verification:* decisions were made by the group and checked against the course handbook.
2. **Report drafting and review.** *Stage:* final revision of the report . *Contribution:* help with a LaTeX compiler error. *Prompt:* "How to fix this red point" *Verification:* check and see if there is any change, especially the statistic number
3. **English language.** *Stage:* writing the notebook narrative, website and report. *Contribution:* sentence-level grammar and clarity edits on text written by the group. *Prompt:* "Fix the grammar and improve the clarity of this paragraph without changing its meaning or any numbers." *Verification:* each edit compared with the original; edits that changed meaning, terminology, or numbers were rejected.
4. **Learning from code examples.** *Stage:* implementation, when a concept or library function was unclear. *Contribution:* small generic examples unrelated to the project pipeline; the project code was written by the group. *Verification:* PyTorch and scikit-learn documentation and small local tests.
5. **Locating course material.** *Stage:* report writing. *Contribution:* with the course slides, the handbook and report drafts loaded as sources, NotebookLM located and quoted the slide passages relevant to statements in the report (for example, metric definitions and preprocessing rules). *Prompt:* "Which slides define macro-F1? Quote the passage and its source." *Verification:* every quoted passage was checked against the original slide or handbook page; the report relies on the original material, not on NotebookLM's summary.
6. **Extension ideas from external references.** *Stage:* after the core requirements were implemented. *Contribution:* pointing to public references and suggesting additional analyses, such as EDA beyond the mandatory items. *Verification:* the group read the original references, selected the ideas to adopt, and implemented and ran every adopted analysis on the dataset.

References

[1] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST: A Novel Image Dataset for Benchmarking Machine Learning Algorithms," *arXiv preprint arXiv:1708.07747*, 2017. https://arxiv.org/abs/1708.07747

[2] A. L. Berger, S. A. Della Pietra, and V. J. Della Pietra, "A maximum entropy approach to natural language processing," *Computational Linguistics*, vol. 22, no. 1, pp. 39–71, 1996. https://aclanthology.org/J96-1002/

[3] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, pp. 533–536, 1986. https://doi.org/10.1038/323533a0

[4] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998. https://doi.org/10.1109/5.726791

[5] T.-S. Le, "Recurrent neural networks: summary" (RNN-Summary, Sections 4–5: training, LSTM and GRU), CO3133 Deep Learning lecture slides, Ho Chi Minh City University of Technology.

[6] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997. https://doi.org/10.1162/neco.1997.9.8.1735

[7] A. Vaswani et al., "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, pp. 5998–6008, 2017. https://proceedings.neurips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html

[8] X. Glorot, A. Bordes, and Y. Bengio, "Deep sparse rectifier neural networks," in *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, vol. 15, pp. 315–323, 2011. https://proceedings.mlr.press/v15/glorot11a.html

[9] A. Dosovitskiy et al., "An image is worth 16x16 words: Transformers for image recognition at scale," in *International Conference on Learning Representations (ICLR)*, 2021. https://openreview.net/forum?id=YicbFdNTTy